Python para Backend

Asincronía, WebSockets y Sistemas Reactivos

Javier Villegas

14 de mayo 2026

Asincronía, WebSockets y Sistemas Reactivos

QUÉ CONSTRUIREMOS EN ESTA SESIÓN

Al terminar tendremos el sistema de encomiendas con capacidades en tiempo real:

1. Notificaciones push: cuando una encomienda cambia de estado, todos los clientes conectados reciben la actualizacion al instante.

2. Dashboard live: los contadores (activas, en transito, con retraso) se actualizan automaticamente sin recargar la pagina.

3. Progreso de tareas: el endpoint bulk_create reporta el avance de creacion masiva en tiempo real.

4. Feed de actividad: cualquier empleado conectado ve todos los cambios del sistema en un feed en vivo.

Asincronía y WebSockets

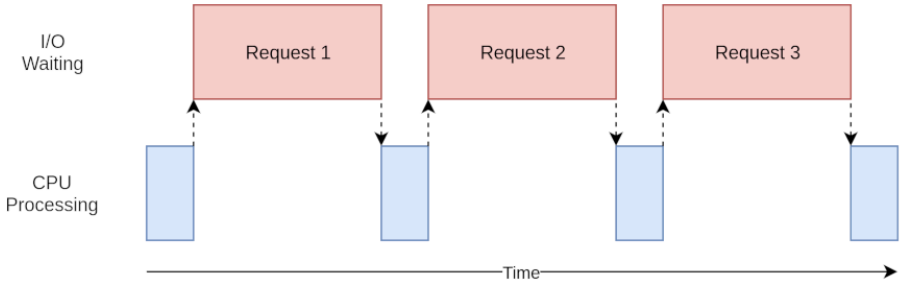
Programación Síncrona vs Asíncrona

En la programación síncrona, cada operación bloquea la ejecución hasta completarse. En la programación asíncrona, mientras una operación espera (BD, red, archivo), Python puede atender otras solicitudes. **En programación, la diferencia principal radica en el orden y el bloqueo de tareas: el código síncrono ejecuta instrucciones secuencialmente (una tras otra), bloqueando el flujo hasta finalizar, mientras que el asíncrono permite iniciar tareas independientes sin esperar a que terminen, mejorando la eficiencia y evitando bloqueo**

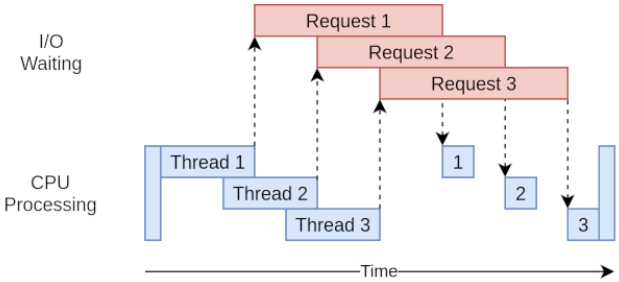
La programación asíncrona es un paradigma de programación que permite una mejor concurrencia, es decir, la ejecución simultánea de múltiples subprocesos. En Python, el módulo `asyncio` ofrece esta capacidad. Varias tareas pueden ejecutarse simultáneamente en un único subproceso, que se programa en un único núcleo de la CPU.

Aunque Python admite el multithreading, la concurrencia está limitada por el Global Interpreter Lock (GIL). El GIL garantiza que solo un hilo pueda adquirir el bloqueo a la vez. La programación asíncrona no resuelve la limitación del GIL, pero permite una mejor concurrencia.

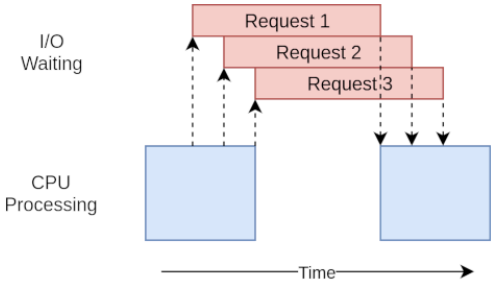
Con el multiprocesamiento, la programación de tareas la realiza el sistema operativo. Con el multihilo, el intérprete de Python se encarga de la programación. En la programación asíncrona de Python, la programación la realiza lo que se denomina el bucle de eventos. Los desarrolladores pueden especificar en su código cuándo una tarea cede voluntariamente la CPU para que el bucle de eventos pueda programar otra tarea. Por esta razón, también se denomina multitarea cooperativa.



(a) Single-threaded process



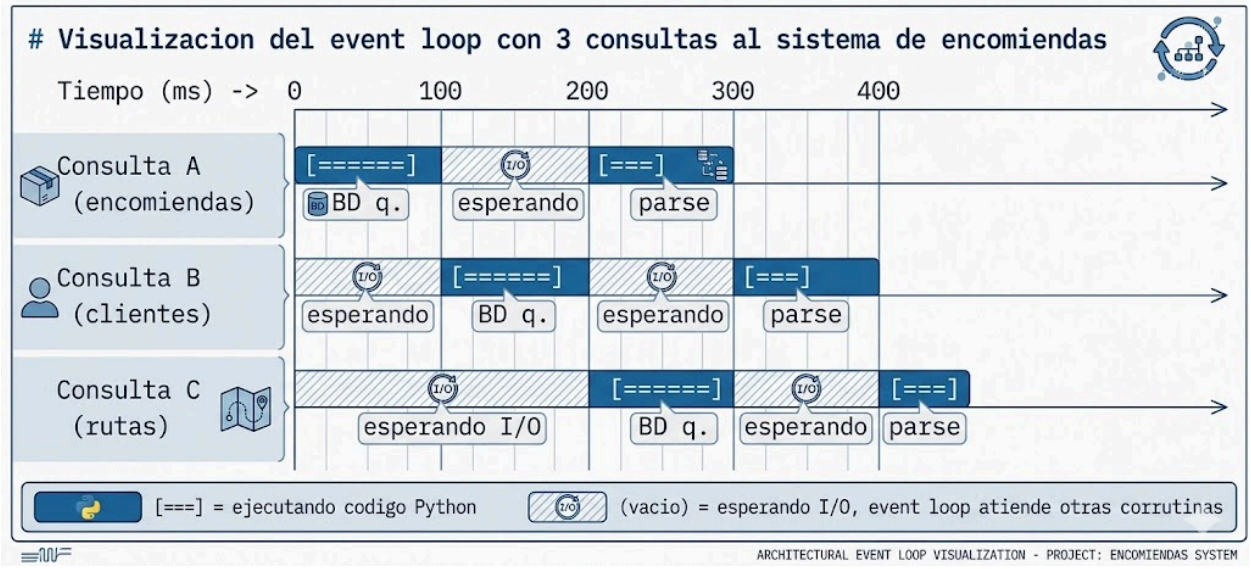
(b) Multi-threaded process
with GIL acquired by current thread



(c) Single-threaded process
with asyncio

El Event Loop — el motor de la asincronía

El event loop es un bucle que monitorea qué corrutinas están listas para ejecutarse. Cuando una corrutina llega a un await, cede el control al event loop. El event loop ejecuta otra corrutina disponible y vuelve a la primera cuando su operación de I/O ha terminado




```
# Total tiempo sincrono:   A + B + C = 300ms + 300ms + 300ms = 900ms
# Total tiempo asincrono:  max(A,B,C) = 300ms
```

None

```
import asyncio

async def consultar_encomiendas():
       # await le dice al event loop: 'pausa aqui y atiende a otros'
       await asyncio.sleep(0.3)   # simula la query a la BD
       return 'encomiendas ok'

async def consultar_clientes():
       await asyncio.sleep(0.3)
       return 'clientes ok'
```

```
async def consultar_rutas():
       await asyncio.sleep(0.3)
       return 'rutas ok'

async def main():
       # gather las lanza TODAS A LA VEZ y espera que terminen
       enc, cli, rut = await asyncio.gather(
       consultar_encomiendas(),
       consultar_clientes(),
       consultar_rutas(),
       )
       print(enc, cli, rut)

asyncio.run(main())  # crea el event loop, ejecuta main, lo cierra
```

Corrutinas — funciones que pueden pausarse

Una corrutina es una función declarada con `async def`. A diferencia de una función normal, una corrutina puede suspenderse en puntos específicos (marcados con `await`) y retomar la ejecución exactamente donde se dejó.

Diferencia fundamental

Python
Función normal vs corrutina

```python
# ── Función normal ──────────────────────────────────────────────
def obtener_encomienda_sync(codigo: str):
       import time
       time.sleep(0.5)     # BLOQUEA: nadie más puede ejecutarse
       return Encomienda.objects.get(codigo=codigo)

# Llamada:
enc = obtener_encomienda_sync('ENC-2026-001')
```

```python
# El hilo esta BLOQUEADO 500ms. Nada más puede ejecutarse.


# ── Corrutina ──────────────────────────────────────────────────────
async def obtener_encomienda_async(codigo: str):
        # await: el event loop puede atender otros requests mientras espera
        enc = await Encomienda.objects.aget(codigo=codigo)
        return enc

# Llamada (solo funciona desde dentro de una funcion async):
enc = await obtener_encomienda_async('ENC-2026-001')
# El event loop CEDE el control mientras espera la BD.
# Otros requests pueden procesarse durante ese tiempo.


# ── Llamar una corrutina desde codigo sincrono ────────────────────
import asyncio
enc = asyncio.run(obtener_encomienda_async('ENC-2026-001'))

# asyncio.run() crea un event loop temporal, ejecuta la corrutina y lo cierra
```

Corrutina completa del proyecto

Python

```python
envios/async_services.py — corrutinas del proyecto
# envios/async_services.py  (nuevo archivo para servicios async)
import asyncio
import httpx
from django.utils import timezone

async def verificar_estado_transportista(codigo: str) -> dict:
        """
        Corrutina que consulta la API del transportista.
        Puede pausarse mientras espera la respuesta HTTP.
        """
```

```
    url = f'https://api.transportista.pe/v1/track/{codigo}'
    try:
    async with httpx.AsyncClient() as client:
        # await: se pausa aqui. El event loop atiende otros requests.
        response = await client.get(url, timeout=5.0)
        data = response.json()
        return {
        'codigo':    codigo,
        'encontrado': True,
        'estado_ext': data.get('status'),
        'ubicacion':  data.get('location'),
        'timestamp':  timezone.now().isoformat(),
        }
    except httpx.TimeoutException:
    return {'codigo': codigo, 'encontrado': False, 'error': 'timeout'}
    except httpx.ConnectError:
    return {'codigo': codigo, 'encontrado': False, 'error': 'conexion'}


async def actualizar_estados_en_transito() -> list:
    """
    Actualiza el estado de todas las encomiendas en transito
    consultando la API del transportista en paralelo.
    """
    # 1. Obtener encomiendas en transito (query async)
    encomiendas = await Encomienda.objects.en_transito().alist()

    if not encomiendas:
    return []

    # 2. Consultar el transportista para TODAS en paralelo
    #    Sin async: 50 enc * 1s = 50 segundos
    #    Con async: ~1 segundo (todas en paralelo)
    resultados = await asyncio.gather(
     *[verificar_estado_transportista(enc.codigo) for enc in encomiendas],
    return_exceptions=True
    )

    # 3. Procesar los resultados
    actualizadas = []
    for enc, resultado in zip(encomiendas, resultados):
    if isinstance(resultado, Exception):
        continue   # ignorar errores individuales
```

```
        if resultado.get('encontrado') and resultado.get('estado_ext') ==
'DELIVERED':
              # La encomienda fue entregada segun el transportista
              enc.estado = 'EN'
              enc.fecha_entrega_real = timezone.now().date()
              await enc.asave()   # guardar async
              actualizadas.append(enc.codigo)


        return actualizadas
```

await — el punto de suspensión

La palabra clave `await` tiene dos efectos: suspende la corrutina actual y devuelve el control al event loop, y extrae el resultado de la corrutina cuando termina. Sólo puede aparecer dentro de una función declarada con `async def`.

Python

```python
Qué se puede usar con await
# await puede usarse con:

# 1. Otras corrutinas
enc = await obtener_encomienda_async('ENC-001')

# 2. Metodos ORM async de Django 4.1+
enc   = await Encomienda.objects.aget(pk=1)
count = await Encomienda.objects.activas().acount()
await enc.asave()

# 3. Clientes HTTP async (httpx)
response = await client.get('https://api.transportista.pe/track/ENC-001')

# 4. asyncio.sleep (sin bloquear)
await asyncio.sleep(5)   # espera 5s sin bloquear el event loop
```

```python
# 5. asyncio.gather (multiples corrutinas en paralelo)
a, b, c = await asyncio.gather(f1(), f2(), f3())

# 6. asyncio.wait_for (con timeout)
resultado = await asyncio.wait_for(mi_corrutina(), timeout=3.0)

# ── Lo que NO se puede await ──────────────────────────────────

# Funciones normales (no son corrutinas)
# await time.sleep(1)      # ERROR: time.sleep no es awaitable

# Queryset sincrono directo
# await Encomienda.objects.all()  # ERROR: no es awaitable
# (usar Encomienda.objects.alist() o sync_to_async)

# Funciones de models.py no async

# await enc.cambiar_estado('TR', emp)  # ERROR si no tiene async def
```

asyncio.gather — paralelismo en el proyecto

La función `asyncio.gather()` toma múltiples corrutinas y las ejecuta todas a la vez. El result es una lista con los resultados en el mismo orden de los argumentos. Es el equivalente async de hacer varias queries a la BD simultáneamente.

Lab 1	**Dashboard con 4 queries en paralelo** Reemplazar el dashboard sincrono por uno async

Python

```python
envios/views_async.py — dashboard stats
# envios/views_async.py  (nuevo archivo)
import asyncio
```

```python
from django.http import JsonResponse
from django.contrib.auth.decorators import login_required
from django.utils import timezone
from django.utils.decorators import method_decorator
from .models import Encomienda

async def dashboard_stats_async(request):
      """
      Endpoint async que calcula las estadisticas del dashboard.
      ANTES (sincrono): 4 queries secuenciales = 4 * 10ms = 40ms
      AHORA (async):      4 queries en paralelo  = max(10ms) = 10ms
      """
      if not request.user.is_authenticated:
      from django.http import HttpResponse
      return HttpResponse(status=401)

      hoy = timezone.now().date()

      # Las 4 queries corren EN PARALELO
      # gather espera a que TODAS terminen
      activas, en_transito, con_retraso, entregadas_hoy = await asyncio.gather(
       Encomienda.objects.activas().acount(),
       Encomienda.objects.en_transito().acount(),
       Encomienda.objects.con_retraso().acount(),
      Encomienda.objects.filter(
            estado='EN', fecha_entrega_real=hoy
      ).acount(),
      )

      return JsonResponse({
      'activas':   activas,
      'en_transito':   en_transito,
      'con_retraso':   con_retraso,
      'entregadas_hoy': entregadas_hoy,

      })
```

Caso complejo: verificar 50 encomiendas en la API del transportista

Python

envios/async_services.py — verificacion masiva en paralelo

```python
# envios/async_services.py
import asyncio
import httpx
from .models import Encomienda

async def verificar_una(session: httpx.AsyncClient, codigo: str) -> dict:
      """Verifica UNA encomienda. Se ejecuta en paralelo con las demas."""
      try:
      r = await session.get(
           f'https://api.transportista.pe/track/{codigo}',
            timeout=5.0
      )
      return {'codigo': codigo, 'ok': True, 'data': r.json()}
      except httpx.TimeoutException:
      return {'codigo': codigo, 'ok': False, 'error': 'timeout'}
      except Exception as e:
      return {'codigo': codigo, 'ok': False, 'error': str(e)}


async def verificar_lote_completo() -> dict:
      """
      Verifica TODAS las encomiendas en transito en paralelo.

      SINCRONO:   50 encomiendas * 1s por consulta = 50 SEGUNDOS
      ASINCRONO:  todas en paralelo                 = ~1 SEGUNDO
      """
      # 1. Obtener encomiendas en transito de la BD
      encomiendas = await Encomienda.objects.en_transito().alist()

      if not encomiendas:
      return {'verificadas': 0, 'resultados': []}

      print(f'Verificando {len(encomiendas)} encomiendas en paralelo...')

      # 2. Abrir una sesion HTTP compartida para todas las consultas
      async with httpx.AsyncClient() as session:
      # 3. Lanzar TODAS las consultas a la vez
      tareas = [
            verificar_una(session, enc.codigo)
            for enc in encomiendas
      ]
```

```python
        # gather: las ejecuta en paralelo y espera a que todas terminen
        resultados = await asyncio.gather(*tareas, return_exceptions=True)

        # 4. Separar exitosas de fallidas
        exitosas = [r for r in resultados if isinstance(r, dict) and r['ok']]
        fallidas  = [r for r in resultados if isinstance(r, dict) and not r['ok']]
        errores   = [r for r in resultados if isinstance(r, Exception)]

        return {
        'verificadas': len(encomiendas),
        'exitosas':  len(exitosas),
        'fallidas':  len(fallidas),
        'errores':   len(errores),
        'resultados':  resultados,
        }


# Llamar desde un comando de management o una vista async
# python manage.py shell
# import asyncio
# from envios.async_services import verificar_lote_completo

# asyncio.run(verificar_lote_completo())
```

asyncio.create_task — lanzar en segundo plano

A diferencia de `await` que espera a que una corrutina termine, `asyncio.create_task()` la lanza en segundo plano y continua la ejecución inmediatamente. El resultado se puede obtener más tarde con `await task`. Ideal para notificaciones y operaciones no críticas.

Python

```python
create_task: notificaciones en segundo plano
import asyncio

async def enviar_notificacion_email(enc, nuevo_estado: str):
        """Envia un email de notificacion. Puede tardar 500ms."""
        # Simula el envio del email
        await asyncio.sleep(0.5)
        print(f'Email enviado: {enc.codigo} -> {nuevo_estado}')
```

```python
async def registrar_en_log_externo(enc, estado: str):
    """Registra el cambio en un sistema de logs externo."""
    import httpx
    async with httpx.AsyncClient() as client:
    await client.post(
        'https://logs.empresa.pe/api/encomiendas',
         json={'codigo': enc.codigo, 'estado': estado},
         timeout=3.0
    )


async def cambiar_estado_vista(request, pk: int):
    """
    Vista async que cambia el estado y lanza las notificaciones
    en background sin hacer esperar al cliente.
    """
    enc          = await Encomienda.objects.aget(pk=pk)
    nuevo_estado = request.data.get('estado')

    # Paso 1: cambiar el estado (CRITICO - el cliente espera esto)
    enc.estado = nuevo_estado
    await enc.asave()

    # Paso 2: lanzar notificaciones en BACKGROUND (no criticas)
    # El cliente recibe la respuesta ANTES de que los emails terminen
    asyncio.create_task(
    enviar_notificacion_email(enc, nuevo_estado)
    )
    asyncio.create_task(
    registrar_en_log_externo(enc, nuevo_estado)
    )

    # Esta respuesta llega al cliente inmediatamente
    # Los emails y logs se envian en segundo plano
    return {'ok': True, 'estado': nuevo_estado}


# Diferencia entre await y create_task:

# CON await: espera a que el email termine antes de responder
# await enviar_notificacion_email(enc, nuevo_estado)  # +500ms de latencia
```

```
# CON create_task: responde al cliente y el email se envia despues

# asyncio.create_task(enviar_notificacion_email(enc, nuevo_estado))  # +0ms
```

asyncio.wait_for — timeout en operaciones async

La función `asyncio.wait_for()` ejecuta una corrutina con un tiempo límite. Si la corrutina no termina en ese tiempo, lanza `asyncio.TimeoutError`. Fundamental cuando se llama a APIs externas que pueden tardar demasiado.

Python

```python
wait_for: timeout en llamadas a API externa
import asyncio
import httpx
from .models import Encomienda

async def verificar_con_timeout(enc) -> dict:
       """
       Verifica una encomienda en la API del transportista.
       Si no responde en 3 segundos, devuelve el ultimo estado conocido.
       """
       try:
       # Maximo 3 segundos para la API externa
       resultado = await asyncio.wait_for(
            verificar_api_externa(enc.codigo),
             timeout=3.0
       )
       return resultado

       except asyncio.TimeoutError:
       # La API tardo mas de 3s -> devolver datos de nuestra BD
       return {
              'codigo':    enc.codigo,
              'estado':    enc.get_estado_display(),
              'fuente':    'cache_local',
              'advertencia': 'API del transportista no disponible',
       }
```

```python
async def verificar_lote_con_timeout(codigos: list) -> list:
    """
    Verifica multiples encomiendas, cada una con su propio timeout.
    Las que fallen (timeout, error de red) devuelven datos del cache.
    """
    encomiendas = await Encomienda.objects.filter(
    codigo__in=codigos
    ).alist()

    resultados = await asyncio.gather(
    *[verificar_con_timeout(enc) for enc in encomiendas],
    return_exceptions=True
    )

    return [
    r if not isinstance(r, Exception) else {'error': str(r)}
    for r in resultados

    ]
```

ORM Asíncrono de Django en el Proyecto

Desde Django 4.1, el ORM tiene equivalentes asíncronos de los métodos más usados. El prefijo `a` identifica la versión async: `get()` → `aget()`, `count()` → `acount()`, etc.

Método síncrono	Método asíncrono	Ejemplo en el proyecto
Model.objects.get()	await Model.objects.aget()	await Encomienda.objects.aget(pk=1)
Model.objects.create()	await Model.objects.acreate()	await Encomienda.objects.acreate(...)
Model.objects.filter().first()	await qs.afirst()	await Encomienda.objects.activas().afirst()
queryset.count()	await queryset.acount()	await Encomienda.objects.con_retraso().acount()

queryset.exists()	await queryset.aexists()	await Encomienda.objects.filter(codigo=c).aexists()
obj.save()	await obj.asave()	enc.estado = 'TR'; await enc.asave()
obj.delete()	await obj.adelete()	await enc.adelete()
list(queryset)	await queryset.alist()	await Encomienda.objects.en_transito().alist()
for obj in queryset:	async for obj in queryset:	async for enc in Encomienda.objects.all():

Iterar un queryset asíncrono

Python

```python
Iteración async y sync_to_async
# ── async for: iterar un queryset sin bloquear el event loop ────────
async def procesar_encomiendas_en_transito():
        """
        Itera todas las encomiendas en transito y actualiza las retrasadas.
        async for no bloquea: cada iteracion cede control al event loop.
        """
        encomiendas_retrasadas = []

        async for enc in Encomienda.objects.en_transito().select_related('ruta'):
        if enc.tiene_retraso:   # @property del modelo
             encomiendas_retrasadas.append(enc)

        # Notificar todas las retrasadas en paralelo
        if encomiendas_retrasadas:
        await asyncio.gather(
              *[notificar_retraso(enc) for enc in encomiendas_retrasadas]
        )

        return len(encomiendas_retrasadas)
```

```python
# —— sync_to_async: si Django < 4.1 o el ORM no tiene metodo async ——
from asgiref.sync import sync_to_async

# Decorador: convierte una funcion sincrona en una async
@sync_to_async
def get_encomiendas_activas():
        return list(Encomienda.objects.activas().con_relaciones())

# Uso:
encomiendas = await get_encomiendas_activas()

# Alternativa en linea (sin decorador):
encomiendas = await sync_to_async(
        lambda: list(Encomienda.objects.activas().con_relaciones())

)()
```

Errores Comunes y Cómo Evitarlos

Error	Causa	Solución
SyntaxError: await outside async	Usar await en funcion sin async def	Declarar la funcion con async def
RuntimeError: Event loop is closed	Llamar asyncio.run() dentro de una corrutina	Usar await directamente, no asyncio.run()
SynchronousOnlyOperation	Llamar ORM sync desde contexto async	Usar aget(), acount() o sync_to_async
Task fue destruido pero esta pendiente	create_task() sin guardar la referencia	task = asyncio.create_task(...) y guardarla
La corrutina nunca se ejecuto	Llamar una corrutina sin await ni create_task	Siempre await o create_task una corrutina

Python

```python
Los 5 errores más comunes con async/await
# ── Error 1: ORM síncrono en contexto async ──────────────────────
async def vista_mal(request):
        encs = list(Encomienda.objects.all())  # SynchronousOnlyOperation

async def vista_bien(request):
        encs = await Encomienda.objects.alist()  # correcto


# ── Error 2: await en funcion sincrona ──────────────────────────
def funcion_sync():
        enc = await Encomienda.objects.aget(pk=1)  # SyntaxError

async def funcion_async():
        enc = await Encomienda.objects.aget(pk=1)  # correcto


# ── Error 3: asyncio.run() dentro de una corrutina ──────────────
async def vista(request):
        enc = asyncio.run(obtener_encomienda(1))  # RuntimeError

async def vista_correcta(request):
        enc = await obtener_encomienda(1)  # correcto


# ── Error 4: corrutina sin await ────────────────────────────────
async def vista(request):
        enc = Encomienda.objects.aget(pk=1)  # devuelve corrutina, no objeto
        print(enc)  # <coroutine object aget at 0x...>

async def vista_correcta(request):
        enc = await Encomienda.objects.aget(pk=1)  # objeto real
        print(enc)  # ENC-2026-001 [Pendiente]


# ── Error 5: Task destruida antes de terminar ───────────────────
async def vista_mal(request):
    asyncio.create_task(enviar_email(enc))  # tarea puede cancelarse

# Solucion: guardar la referencia
_tasks = set()
async def vista_bien(request):
        task = asyncio.create_task(enviar_email(enc))
        _tasks.add(task)                 # evitar que el GC la destruya
```

```
task.add_done_callback(_tasks.discard)  # limpiar al terminar
```

RESUMEN — GUÍA RÁPIDA DE ASYNC/AWAIT EN EL PROYECTO

Declarar una corrutina: `async def mi_funcion():`

Esperar una corrutina: `resultado = await mi_corrutina()`

Ejecutar varias en paralelo: `a, b = await asyncio.gather(f1(), f2())`

Lanzar en background: `asyncio.create_task(mi_corrutina())`

Con timeout: `await asyncio.wait_for(f(), timeout=3.0)`

Desde codigo sincrono: `asyncio.run(mi_corrutina())`

ORM async: `await Encomienda.objects.aget(pk=1)`

ORM async count: `await Encomienda.objects.activas().acount()`

ORM async guardar: `enc.estado = 'TR'; await enc.asave()`

ORM async iterar: `async for enc in Encomienda.objects.all():`

Convertir sync a async: `@sync_to_async / sync_to_async(lambda: ...)()`

Archivos nuevos en el proyecto:

envios/async_services.py <- verificar_lote_completo, verificar_con_timeout

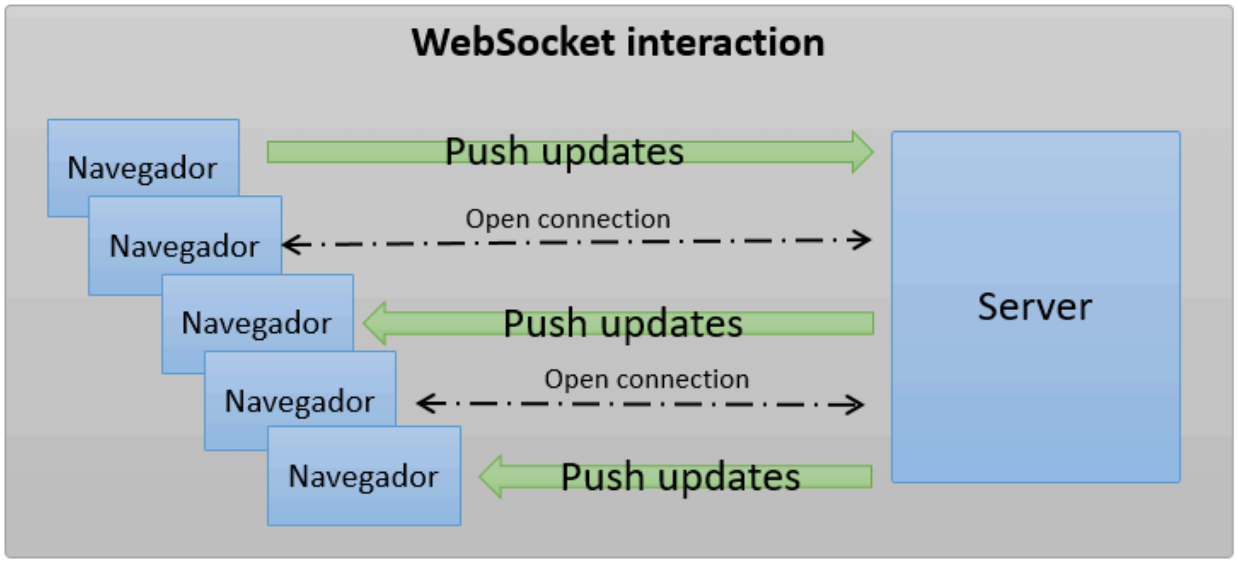
envios/views_async.py <- dashboard_stats_async, cambiar_estado_vista

Introducción a WebSockets

WebSocket es un protocolo de comunicación que permite una conexión bidireccional, persistente y en tiempo real entre un navegador (cliente) y un servidor. A diferencia de HTTP, que requiere una petición por cada respuesta, WebSocket mantiene la conexión abierta, permitiendo el intercambio instantáneo de datos sin necesidad de constantes peticiones nuevas.

Aspectos Clave de WebSocket:

- Comunicación Bidireccional: Tanto el cliente como el servidor pueden enviarse mensajes en cualquier momento.
- Tiempo Real: Es ideal para aplicaciones que requieren actualizaciones inmediatas, como chats, juegos online o marcadores deportivos.
- Conexión Persistente: Se establece un "handshake" inicial y la conexión permanece abierta (full-duplex).
- Eficiencia: Reduce la latencia y la sobrecarga de datos en comparación con HTTP, al no tener que abrir y cerrar conexiones repetidamente.
- Compatibilidad: Funciona sobre los puertos estándar 80 y 443, lo que permite superar cortafuegos



La analogía: telefóno vs walkie-talkie

Para entender WebSockets, la mejor analogía es comparar una llamada telefónica con un intercambio de cartas.

HTTP (cartas)	WebSocket (teléfono)
El cliente escribe y envía una carta (request)	Ambos marcan el número y establecen la llamada
El servidor lee y responde con otra carta	Cualquiera puede hablar en cualquier momento
La carta llega y la comunicación termina	La línea se mantiene abierta mientras dure la sesión
Para saber si hay respuesta, hay que preguntar de nuevo	El servidor puede hablar sin que el cliente lo solicite
Cada carta tiene su propio sobre con dirección (headers)	Solo un «over» al inicio para abrir la línea (handshake)

En el sistema de encomiendas: cuando el empleado Luis cambia el estado de ENC-2026-001 a **En tránsito**, todos los navegadores conectados reciben esa notificación al instante. No hay forma de hacer eso con HTTP puro: necesitaríamos un WebSocket.

El problema: HTTP polling vs WebSocket push

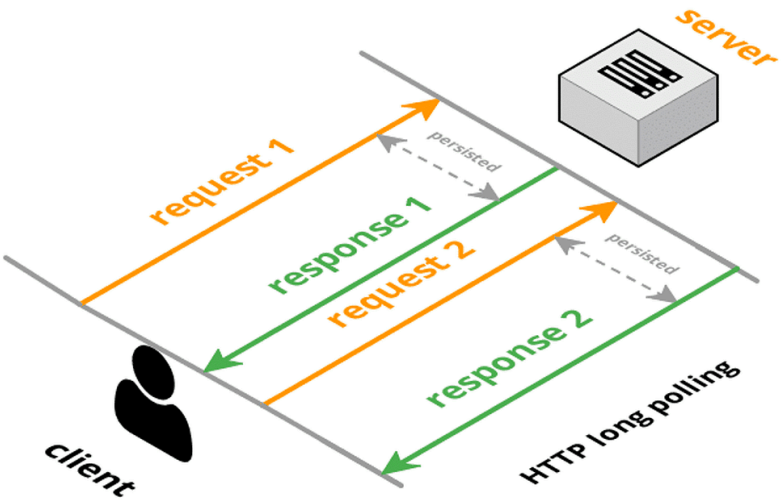
Imagina que el remitente quiere saber en qué estado está su encomienda. Con HTTP tiene dos opciones: preguntar repetidamente (polling) o esperar a que alguien le avise (push con WebSocket).

El problema del polling con HTTP

El polling HTTP (sondeo) es una técnica donde un cliente (como un navegador) pregunta repetidamente a un servidor si hay nuevos datos a intervalos regulares. Aunque es fácil de implementar, presenta serios problemas de eficiencia y rendimiento en aplicaciones modernas en tiempo real

Los principales problemas del polling con HTTP son:

- Desperdicio de Recursos (Red y CPU): El cliente realiza constantes peticiones HTTP, incluso cuando no hay datos nuevos. Esto consume ancho de banda y carga de CPU tanto en el cliente como en el servidor.
- Latencia Artificial: Existe un retraso entre el momento en que los datos cambian en el servidor y el momento en que el cliente realiza la siguiente petición y recibe la actualización. La información no es instantánea.
- Problemas de Escalabilidad: Con muchos usuarios, miles de peticiones vacías por segundo pueden saturar el servidor, afectando su rendimiento.
- Sobrecarga de Cabeceras HTTP: Cada petición lleva consigo cabeceras HTTP, lo que añade datos innecesarios a cada consulta



```python
El problema del HTTP polling
# Lo que pasa con polling HTTP en el sistema de encomiendas:
#
# 10:00:00 - Navegador pregunta: GET /api/v1/encomiendas/1/  -> PE (Pendiente)
# 10:00:05 - Navegador pregunta: GET /api/v1/encomiendas/1/  -> PE (sin cambio)
# 10:00:10 - Navegador pregunta: GET /api/v1/encomiendas/1/  -> PE (sin cambio)
# 10:00:15 - Navegador pregunta: GET /api/v1/encomiendas/1/  -> PE (sin cambio)
# 10:00:18 - Luis cambia el estado a TR en el sistema
# 10:00:20 - Navegador pregunta: GET /api/v1/encomiendas/1/  -> TR (!)
#
# Problemas del polling:
# 1. Demora de hasta 5 segundos para enterarse del cambio
# 2. Con 50 empleados conectados: 50 requests cada 5s = 600 req/min
# 3. La mayoria de esos 600 requests devuelven 'sin cambio' (desperdicio)
# 4. El servidor procesa carga aunque no haya nada que reportar

# Implementacion tipica del polling (JavaScript):
setInterval(async () => {
       const r = await fetch('/api/v1/encomiendas/1/');
       const data = await r.json();
       if (data.estado !== estadoAnterior) {
       actualizarUI(data.estado);
       estadoAnterior = data.estado;
       }
}, 5000);  // cada 5 segundos


# Con WebSocket: 0 requests hasta que algo cambia
```

La solución: WebSocket push

El "WebSocket push" es una técnica de comunicación en tiempo real que utiliza el protocolo WebSocket para enviar datos desde el servidor al cliente (push) de forma instantánea y bidireccional. A diferencia de HTTP, mantiene una conexión abierta, permitiendo actualizaciones inmediatas sin necesidad de que el cliente solicite información repetidamente.

Ventajas clave del WebSocket Push:

- Tiempo Real: Ideal para chats, notificaciones instantáneas, juegos, cotizaciones en vivo y paneles de monitoreo (IoT).
- Eficiencia: Al mantener la conexión abierta (full-duplex), elimina la sobrecarga de cabeceras de HTTP repetitivas, reduciendo el consumo de ancho de banda.
- Baja Latencia: Los mensajes se transmiten al instante en que el servidor los recibe o genera

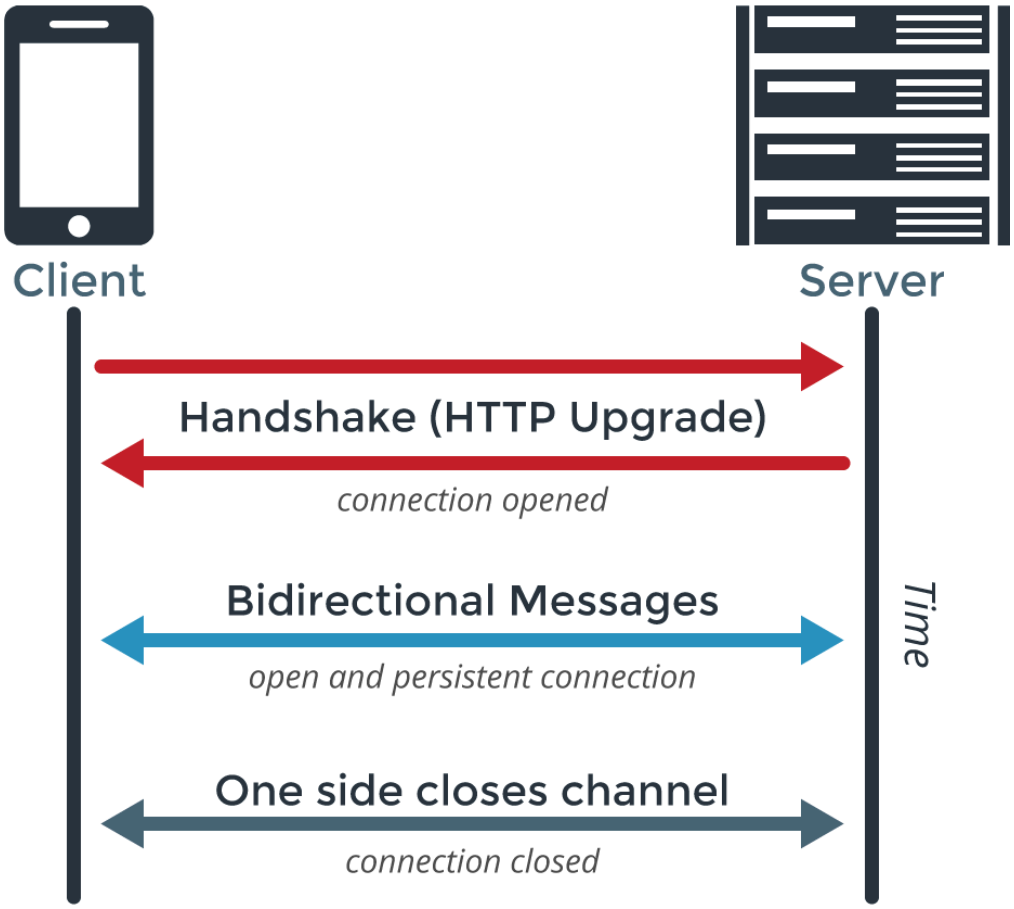
Python

```
La solución con WebSocket push
# Lo que ocurre con WebSocket en el sistema de encomiendas:
#
# 10:00:00 - Empleado abre el navegador
# 10:00:00 - Navegador abre UN WebSocket: ws://localhost/ws/encomiendas/
# 10:00:00 - Servidor acepta la conexion (101 Switching Protocols)
# 10:00:00 - Servidor envia las estadisticas iniciales
#
# ... la conexion esta ABIERTA, no se consume red ...
#
# 10:00:18 - Luis cambia ENC-2026-001 de PE a TR
# 10:00:18 - El modelo llama a channel_layer.group_send()
# 10:00:18 - TODOS los navegadores conectados reciben instantaneamente:
#            {tipo: 'estado_cambio', codigo: 'ENC-2026-001',
#            estado_anterior: 'PE', estado_nuevo: 'TR',
#            empleado: 'Mendoza Cruz, Luis'}
#
# Ventajas:
# 1. Notificacion en <100ms (tiempo de red, no de polling)
# 2. 0 requests adicionales hasta el proximo cambio
# 3. El servidor solo envia cuando hay algo que enviar

# 4. Escala a miles de conexiones con poco CPU
```

El ciclo de vida de una conexión WebSocket

Una conexión WebSocket tiene cuatro fases bien definidas: el handshake para establecerla, la comunicación bidireccional, y el cierre ordenado.



Fase 1: El Handshake — cómo se abre la conexión

El handshake es una petición HTTP especial. El cliente envía una petición `GET` con cabeceras especiales pidiendo cambiar el protocolo. El servidor responde con `101 Switching Protocols` y desde ese momento la conexión TCP se convierte en un canal WebSocket.

```
Python
```

```
El handshake paso a paso
# ── PASO 1: El cliente (navegador) envia una peticion HTTP normal ─
# pero con cabeceras especiales de WebSocket:

GET /ws/encomiendas/ HTTP/1.1
Host: localhost:8000
Upgrade: websocket         <- pide cambiar el protocolo
Connection: Upgrade        <- confirma que quiere hacer upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==   <- clave aleatoria base64
Sec-WebSocket-Version: 13   <- version del protocolo
Origin: http://localhost:8000

# ── PASO 2: El servidor acepta y responde 101 ────────────────────

HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
# El Accept es SHA-1 de la Key del cliente + magic string

# ── PASO 3: La conexion ya no es HTTP ────────────────────────────
# La conexion TCP queda abierta y habla WebSocket frames.
# Django Channels llama a connect() del consumidor:

# envios/consumers.py
class EncomiendaConsumer(AsyncWebsocketConsumer):
       async def connect(self):
       user = self.scope['user']
       if not user.is_authenticated:
              await self.close(code=4001)  # rechazo personalizado
              return

       # Unirse al grupo global para recibir todas las notificaciones
       await self.channel_layer.group_add('encomiendas_global', self.channel_name)
       await self.accept()  # <- confirma la conexion con el cliente

       # Enviar mensaje de bienvenida con estadisticas actuales
       stats = await self.get_estadisticas()
       await self.send(text_data=json.dumps({
              'tipo':      'conectado',
              'mensaje': f'Bienvenido, {user.username}',
              'stats':   stats,

       }))
```

Fase 2: Comunicación bidireccional

Una vez establecida la conexión, cliente y servidor pueden enviarse mensajes en **cualquier dirección** en **cualquier momento**, sin esperar a que el otro lo solicite. Los mensajes viajan en **frames** WebSocket, que son muchío más pequeños que una cabecera HTTP.

Python

```
Comunicación bidireccional
# ── Mensajes del cliente al servidor (receive) ──────────────────

# El cliente envia un ping para verificar que la conexion sigue activa
ws.send(JSON.stringify({tipo: 'ping'}))

# El servidor responde en receive():
async def receive(self, text_data):
       data = json.loads(text_data)
       if data['tipo'] == 'ping':
       await self.send(text_data=json.dumps({'tipo': 'pong'}))


# ── Mensajes del servidor al cliente (push) ────────────────────

# Cuando Luis cambia el estado de ENC-2026-001:
# 1. El modelo llama a _notificar_cambio_estado()
# 2. Esa funcion usa el channel layer para publicar en el grupo
# 3. Channels distribuye el mensaje a TODOS los consumidores del grupo
# 4. Cada consumidor llama a su handler y envia al WebSocket:

async def encomienda_estado_cambio(self, event):
       """Handler: recibe del channel layer y reenvia al navegador"""
       await self.send(text_data=json.dumps({
       'tipo':             'estado_cambio',
       'encomienda_id':   event['encomienda_id'],
       'codigo':           event['codigo'],
       'estado_anterior': event['estado_anterior'],
       'estado_nuevo':     event['estado_nuevo'],
       'empleado':         event['empleado'],
       'timestamp':        event['timestamp'],
       }))

# 5. El navegador recibe el mensaje en onmessage:
```

```
ws.onmessage = function(event) {
       const data = JSON.parse(event.data);
       if (data.tipo === 'estado_cambio') {
       mostrarToast(data.codigo, data.estado_anterior, data.estado_nuevo);
       }

}
```

Fase 3: El cierre ordenado

Cualquier extremo puede iniciar el cierre. Se envía un **frame de cierre** con un código y razón. El protocolo especifica códigos numerados: 1000 = cierre normal, 1001 = el usuario se fue, 4001 = no autorizado (códigos 4000-4999 son personalizados).

Python

```
Cierre de conexión
# ── Cierre desde el cliente (JavaScript) ──────────────────────────

// Cierre normal al navegar a otra pagina
ws.close(1000, 'Usuario cerro la pestana');

// El servidor recibe el cierre en disconnect():
async def disconnect(self, close_code):
       """Se llama cuando el cliente cierra la conexion"""
       print(f'Cliente desconectado con codigo: {close_code}')
       # Salir del grupo para no recibir mas mensajes
       await self.channel_layer.group_discard(
       'encomiendas_global',
       self.channel_name
       )


# ── Cierre desde el servidor (Django Channels) ────────────────────

# Cerrar por inactividad o falta de autorizacion:
await self.close(code=4001)  # codigo personalizado: no autorizado
```

```
# El cliente recibe el cierre en onclose:
ws.onclose = function(event) {
      console.log(`Cerrado. Codigo: ${event.code}`);
      if (event.code === 4001) {
      // El servidor rechazo la conexion (no autenticado)
      window.location.href = '/accounts/login/';
      } else if (event.code !== 1000) {
      // Desconexion inesperada: reconectar en 3 segundos
      setTimeout(() => location.reload(), 3000);
      }

};
```

Frames WebSocket — cómo viajan los mensajes

A diferencia de HTTP donde cada mensaje lleva cientos de bytes de cabeceras, los mensajes WebSocket viajan en frames que añaden entre 2 y 14 bytes de overhead. Eso los hace ideales para mensajes frecuentes como notificaciones.

Tipo de frame	Uso en el proyecto
Text frame (0x1)	Mensajes JSON del sistema (estado_cambio, stats, ping/pong)
Binary frame (0x2)	No usado en el proyecto (para imágenes o archivos binarios)
Close frame (0x8)	Cierre de conexión con código y razón
Ping frame (0x9)	Django Channels envía pings automáticos para mantener la conexión
Pong frame (0xA)	El cliente responde automáticamente a los pings del servidor

Python

```
Estructura interna de un frame WebSocket
```

```
# Estructura de un frame WebSocket (simplificada):
#
# Byte 0: FIN (1 bit) + Opcode (4 bits)
#   FIN = 1: este es el frame final del mensaje
#   Opcode 0x1 = texto, 0x8 = close, 0x9 = ping, 0xA = pong
#
# Byte 1: MASK bit (1 bit) + Payload length (7 bits)
#   Los frames del cliente SIEMPRE van enmascarados
#   Los frames del servidor NO van enmascarados
#
# Bytes 2-9: Extended length (si el mensaje es largo)
# Bytes siguientes: Masking key (4 bytes, solo si MASK=1)
# Resto: Payload (datos reales)
#
# Ejemplo: mensaje JSON '{"tipo":"ping"}' (13 caracteres)
# Overhead HTTP:    ~400-600 bytes de cabeceras
# Overhead WS:      2-6 bytes

# El frame WS es 100x mas eficiente para mensajes cortos frecuentes
```

La API JavaScript del WebSocket

El navegador tiene la clase `WebSocket` nativa. No requiere instalar ninguna librería. Solo se necesita la URL del endpoint WebSocket del servidor.

Los 4 eventos del WebSocket

Python

```
Los 4 eventos del WebSocket (JavaScript completo)
const ws = new WebSocket('ws://localhost:8000/ws/encomiendas/');
// wss:// para HTTPS/SSL (siempre en produccion)
```

```
// ── onopen: conexion establecida ──────────────────────────────
ws.onopen = function(event) {
       // Se dispara cuando el handshake termina exitosamente
       // ws.readyState === 1 (OPEN)
       console.log('Conectado!');

       // Mostrar el indicador 'EN VIVO' en el navbar
    document.getElementById('ws-badge').textContent = 'EN VIVO';
    document.getElementById('ws-badge').classList.add('text-success');

       // Pedir las estadisticas actuales al conectarse
       ws.send(JSON.stringify({ tipo: 'solicitar_stats' }));
};

// ── onmessage: mensaje recibido del servidor ──────────────────
ws.onmessage = function(event) {
       // event.data: string JSON con los datos del mensaje
       const data = JSON.parse(event.data);

       switch(data.tipo) {
       case 'conectado':
              actualizarDashboard(data.stats);
              break;

       case 'estado_cambio':
              // Un empleado cambio el estado de una encomienda
              mostrarNotificacion(
              data.codigo,
              data.estado_anterior,
              data.estado_nuevo,
              data.empleado,
              data.timestamp
              );
              actualizarFilaTabla(data.codigo, data.estado_nuevo);
              break;

       case 'stats_actualizado':
              // El dashboard se actualiza automaticamente
              actualizarDashboard(data.stats);
              break;

       case 'progreso':
              // Progreso del bulk_create
```

```
            const pct = Math.round(data.actual / data.total * 100);
            actualizarBarra(pct, data.codigo);
            break;
      }
};

// ── onclose: conexion cerrada ──────────────────────────────────────
ws.onclose = function(event) {
      // event.code: codigo de cierre (1000=normal, 4001=no autorizado)
      // event.reason: texto descriptivo del motivo
      // event.wasClean: true si fue un cierre limpio
      console.log(`Cerrado con codigo ${event.code}: ${event.reason}`);

    document.getElementById('ws-badge').textContent = 'Desconectado';
    document.getElementById('ws-badge').classList.remove('text-success');

      if (event.code === 4001) {
      // No autorizado: redirigir al login
      window.location.href = '/accounts/login/';
      } else if (event.code !== 1000 && event.code !== 1001) {
      // Desconexion inesperada: reconectar en 3 segundos
      console.log('Reconectando...');
      setTimeout(() => {
            const nuevoWs = new WebSocket(ws.url);
            // reasignar handlers...
      }, 3000);
      }
};

// ── onerror: error de red ──────────────────────────────────────────
ws.onerror = function(error) {
      // Se dispara antes de onclose en caso de error
      // No tiene mucha informacion (seguridad del navegador)
      console.error('Error WebSocket:', error);

};
```

Códigos de cierre

Código	Nombre	Cuándo ocurre en el sistema de encomiendas

1000	Normal closure	El empleado cerró sesión voluntariamente
1001	Going away	El empleado cerró la pestaña o naegó a otra página
1006	Abnormal closure	Perdió la conexión a internet (sin frame de cierre)
1011	Internal error	Error no controlado en un consumer de Channels
4001	No autorizado	El usuario no está autenticado (código personalizado)
4002	Sesión expirada	El token JWT expiró (código personalizado)

Verificar la conexión desde la consola del navegador

La mejor forma de entender WebSockets es conectarse directamente desde las herramientas de desarrollo del navegador. No necesitas nada instalado.

Lab	**Probar el WebSocket del sistema desde el navegador** Abrir http://localhost:8000 con el sistema corriendo

Python

```
Prueba desde la consola del navegador (paso a paso)
# 1. Abrir el sistema en el navegador: http://localhost:8000
# 2. Iniciar sesión como superusuario
# 3. Abrir las herramientas de desarrollo: F12
# 4. Ir a la pestaña 'Console'
# 5. Escribir el siguiente codigo:

// Conectar al WebSocket
const ws = new WebSocket('ws://localhost:8000/ws/encomiendas/');

// Ver todos los mensajes
ws.onopen    = () => console.log('Conectado al sistema!');
ws.onmessage = e => console.log('Mensaje recibido:', JSON.parse(e.data));
ws.onclose   = e => console.log('Cerrado:', e.code, e.reason);
ws.onerror   = e => console.error('Error:', e);
```

```
// Resultado esperado en la consola:
// 'Conectado al sistema!'
// Mensaje recibido: {tipo: 'conectado', mensaje: 'Bienvenido, admin',
//                   stats: {activas: 50, en_transito: 20, ...}}

// Enviar un ping al servidor:
ws.send(JSON.stringify({tipo: 'ping'}));
// Resultado: {tipo: 'pong'}

// Pedir estadisticas:
ws.send(JSON.stringify({tipo: 'solicitar_stats'}));
// Resultado: {tipo: 'stats', stats: {activas: 50, ...}}

// Ahora, en OTRA pestana del navegador:
// - Ir a Swagger: http://localhost:8000/api/docs/
// - Autenticarse con el token JWT
// - Ejecutar: POST /api/v1/encomiendas/1/cambiar_estado/
//   Body: {"estado": "TR", "observacion": "Recogido en agencia"}
//
// De vuelta en la primera pestana, en la consola veras:
// Mensaje recibido: {tipo: 'estado_cambio', codigo: 'ENC-2026-001',
//                   estado_anterior: 'PE', estado_nuevo: 'TR',
//                   empleado: 'admin', timestamp: '...'  }

// Ver la conexion en la pestana Network:
// F12 -> Network -> WS (filtrar por WebSocket)
// Hacer clic en la conexion ws://localhost:8000/ws/encomiendas/

// En la pestaña 'Messages' se ven todos los frames enviados y recibidos
```

RESUMEN — CONCEPTOS CLAVE DE WEBSOCKETS

Qué es: protocolo de comunicación bidireccional y persistente sobre TCP.

Diferencia con HTTP:

HTTP: el cliente pregunta, el servidor responde, la conexion se cierra.

WebSocket: la conexion queda abierta, cualquiera puede enviar mensajes.

El handshake:

1. Cliente envia GET con Upgrade: websocket
2. Servidor responde 101 Switching Protocols
3. La conexion TCP ya no habla HTTP, habla WebSocket frames
4. Django Channels llama a connect() del consumer

Los 4 eventos del cliente JavaScript:

onopen: la conexion se estableció correctamente

onmessage: llego un mensaje del servidor (event.data = JSON string)

onclose: la conexion se cerro (event.code indica el motivo)

onerror: error de red (seguido siempre de onclose)

En el proyecto de encomiendas:

ws://localhost:8000/ws/encomiendas/ <- notificaciones globales

ws://localhost:8000/ws/encomiendas/{pk}/ <- una encomienda especifica

ws://localhost:8000/ws/dashboard/ <- estadisticas en tiempo real

Ejemplo Completo: Dashboard en Tiempo Real

Este ejemplo integra todos los conceptos vistos: el handshake, los eventos del WebSocket, el consumer de Django Channels y la notificación desde el modelo. Implementa el dashboard del sistema de encomiendas con actualización automática de contadores y un feed de actividad en vivo.

QUÉ HACE ESTE EJEMPLO

1. El navegador abre una conexion WebSocket al cargar el dashboard.
2. El servidor (Django Channels) acepta la conexion y envia las estadisticas iniciales.
3. Cuando cualquier empleado cambia el estado de una encomienda (desde la API REST o la web), el modelo notifica al channel layer.
4. Channels distribuye la notificacion a TODOS los navegadores conectados.
5. Cada navegador actualiza los contadores y muestra un toast sin recargar la pagina.

Paso 1 — El template del dashboard (HTML + JavaScript)

Este archivo reemplaza el dashboard sincrono. Abre `templates/envios/dashboard.html` y agrega el bloque `extra_js`:

Python

```
templates/envios/dashboard.html — estructura HTML
{% extends 'base.html' %}

{% block content %}
<!-- Indicador de conexion en el navbar -->
<div class="d-flex justify-content-between align-items-center mb-4">
```

```
  <h2>Dashboard</h2>
  <span id="ws-badge" class="badge bg-secondary">Conectando...</span>
</div>

<!-- Tarjetas de estadisticas -->
<div class="row g-3 mb-4">
  <div class="col-md-3">
       <div class="card shadow-sm">
       <div class="card-body text-center">
       <div class="fs-1">&#128230;</div>
       <div class="fs-2 fw-bold text-primary" id="stat-activas">{{ stats.activas
}}</div>
       <div class="text-muted">Activas</div>
       </div>
       </div>
  </div>
  <div class="col-md-3">
       <div class="card shadow-sm">
       <div class="card-body text-center">
       <div class="fs-1">&#128666;</div>
       <div class="fs-2 fw-bold text-warning" id="stat-en-transito">{{
stats.en_transito }}</div>
       <div class="text-muted">En tránsito</div>
       </div>
       </div>
  </div>
  <div class="col-md-3">
       <div class="card shadow-sm">
       <div class="card-body text-center">
       <div class="fs-1">&#9888;</div>
       <div class="fs-2 fw-bold text-danger" id="stat-retraso">{{ stats.con_retraso
}}</div>
       <div class="text-muted">Con retraso</div>
       </div>
       </div>
  </div>
  <div class="col-md-3">
       <div class="card shadow-sm">
       <div class="card-body text-center">
       <div class="fs-1">&#9989;</div>
       <div class="fs-2 fw-bold text-success" id="stat-entregadas">{{
stats.entregadas_hoy }}</div>
       <div class="text-muted">Entregadas hoy</div>
       </div>
```

```
      </div>
  </div>
</div>

<!-- Feed de actividad en tiempo real -->
<div class="card shadow-sm">
  <div class="card-header d-flex justify-content-between">
      <span>Feed de actividad</span>
      <small class="text-muted" id="feed-count">0 eventos</small>
  </div>
  <ul class="list-group list-group-flush" id="feed-lista">
      <li class="list-group-item text-muted">Esperando eventos...</li>
  </ul>
</div>

{% endblock %}
```

Python

templates/envios/dashboard.html — bloque extra_js completo

```
{% block extra_js %}
<script>
// ── Configuracion del WebSocket ──────────────────────────────
const WS_URL = 'ws://' + window.location.host + '/ws/dashboard/';
let ws;
let eventoCount = 0;

const ESTADOS = {
  PE: 'Pendiente', TR: 'En tránsito',
  DE: 'En destino', EN: 'Entregado', DV: 'Devuelto'
};
const COLORES_BADGE = {
  PE: 'secondary', TR: 'primary',
  DE: 'warning',   EN: 'success', DV: 'danger'
};
const ICONOS = {
  PE: '⏳', TR: '🚚', DE: '📍', EN: '✅', DV: '↩'
};

// ── Conectar al WebSocket del dashboard ──────────────────────
```

```
function conectarWebSocket() {
  ws = new WebSocket(WS_URL);

  ws.onopen = function() {
       console.log('WebSocket conectado al dashboard');
       const badge = document.getElementById('ws-badge');
       badge.textContent = 'EN VIVO';
       badge.className = 'badge bg-success';
  };

  ws.onmessage = function(event) {
       const data = JSON.parse(event.data);

       // Actualizar contadores del dashboard
       if (data.tipo === 'stats_iniciales' || data.tipo === 'stats_actualizado') {
       actualizarContador('stat-activas',      data.stats.activas);
       actualizarContador('stat-en-transito', data.stats.en_transito);
       actualizarContador('stat-retraso',      data.stats.con_retraso);
       actualizarContador('stat-entregadas', data.stats.entregadas_hoy);
       }

       // Agregar evento al feed de actividad
       if (data.tipo === 'estado_cambio') {
       agregarAlFeed(data);
       mostrarToast(data);
       }
  };

  ws.onclose = function(event) {
       const badge = document.getElementById('ws-badge');
       badge.textContent = 'Desconectado';
       badge.className = 'badge bg-danger';
       console.log('WebSocket cerrado, codigo:', event.code);
       // Reconectar si fue una desconexion inesperada
       if (event.code !== 1000) {
       console.log('Reconectando en 3 segundos...');
       setTimeout(conectarWebSocket, 3000);
       }
  };

  ws.onerror = function(error) {
       console.error('Error en el WebSocket del dashboard:', error);
  };
}
```

```
// ── Funciones auxiliares ──────────────────────────────────────────
function actualizarContador(id, nuevoValor) {
  const el = document.getElementById(id);
  if (!el) return;
  const valorAnterior = parseInt(el.textContent);
  if (valorAnterior === nuevoValor) return;
  // Animacion de resaltado al cambiar
  el.style.transition = 'transform 0.2s';
  el.style.transform  = 'scale(1.4)';
  el.textContent    = nuevoValor;
  setTimeout(() => { el.style.transform = 'scale(1)'; }, 250);
}

function agregarAlFeed(data) {
  eventoCount++;
  document.getElementById('feed-count').textContent = eventoCount + ' eventos';

  const lista = document.getElementById('feed-lista');

  // Quitar el mensaje inicial si es el primer evento
  if (eventoCount === 1) lista.innerHTML = '';

  const hora = new Date(data.timestamp).toLocaleTimeString('es-PE');
  const li = document.createElement('li');
  li.className = 'list-group-item d-flex align-items-center gap-2 py-2
animate__animated animate__fadeInDown';
  li.innerHTML = `
      <span class='fs-5'>${ICONOS[data.estado_nuevo] || ''}</span>
      <div class='flex-grow-1'>
     <strong>${data.codigo}</strong>
      <span class='text-muted'> — </span>
      <span class='badge
bg-${COLORES_BADGE[data.estado_anterior]}'>${ESTADOS[data.estado_anterior]}</span>
      →
      <span class='badge
bg-${COLORES_BADGE[data.estado_nuevo]}'>${ESTADOS[data.estado_nuevo]}</span>
      <small class='d-block text-muted'>Por: ${data.empleado} &bull;
${hora}</small>
      </div>
  `;
  // Insertar al inicio de la lista
  lista.insertBefore(li, lista.firstChild);
  // Limitar a 20 eventos en pantalla
```

```
  while (lista.children.length > 20) lista.removeChild(lista.lastChild);
}

function mostrarToast(data) {
  const container = document.getElementById('toast-container')
       || crearToastContainer();
  const toast = document.createElement('div');
  toast.className = `alert alert-${COLORES_BADGE[data.estado_nuevo]}
alert-dismissible fade show`;
  toast.style.cssText = 'min-width: 300px; box-shadow: 0 2px 8px
rgba(0,0,0,.15);';
  toast.innerHTML = `
       <div class='d-flex align-items-center gap-2'>
       <span class='fs-4'>${ICONOS[data.estado_nuevo]}</span>
       <div>
        <strong>${data.codigo}</strong><br>
       ${ESTADOS[data.estado_anterior]} &rarr; ${ESTADOS[data.estado_nuevo]}<br>
       <small>Por: ${data.empleado}</small>
       </div>
       </div>
       <button type='button' class='btn-close' data-bs-dismiss='alert'></button>
  `;
  container.appendChild(toast);
  setTimeout(() => toast.remove(), 5000);
}

function crearToastContainer() {
  const div = document.createElement('div');
  div.id = 'toast-container';
  div.style.cssText =
'position:fixed;top:80px;right:20px;z-index:9999;display:flex;flex-direction:colum
n;gap:8px;';
  document.body.appendChild(div);
  return div;
}

// ── Iniciar la conexion al cargar la pagina ──────────────────────
document.addEventListener('DOMContentLoaded', conectarWebSocket);
</script>

{% endblock %}
```

Paso 2 — Vista del dashboard que pasa datos iniciales

Abre `envios/views.py` y agrega la vista del dashboard. Pasa las estadísticas iniciales al template para que los contadores no aparezcan en 0 mientras llega el WebSocket:

Python

```python
envios/views.py — vista del dashboard
# envios/views.py
from django.contrib.auth.decorators import login_required
from django.shortcuts import render
from django.utils import timezone
from .models import Encomienda

@login_required
def dashboard(request):
    """
    El template renderiza con los datos iniciales de la BD.
    El WebSocket actualiza los contadores en tiempo real a partir de ese punto.
    """
    hoy = timezone.now().date()
    context = {
    'stats': {
        'activas':   Encomienda.objects.activas().count(),
        'en_transito':   Encomienda.objects.en_transito().count(),
        'con_retraso':   Encomienda.objects.con_retraso().count(),
        'entregadas_hoy': Encomienda.objects.filter(
        estado='EN', fecha_entrega_real=hoy
        ).count(),
    }
    }

    return render(request, 'envios/dashboard.html', context)
```

Paso 3 — Consumer del dashboard (ya implementado en sección anterior)

El consumer `DashboardConsumer` en `envios/consumers.py` ya implementado en la sección 7.6 recibe los mensajes `dashboard_actualizar` del channel layer y los envía al navegador. Aquí se muestra cómo envía los stats iniciales al conectarse:

Python

```python
envios/consumers.py — DashboardConsumer (referencia)
# envios/consumers.py
# DashboardConsumer ya implementado en seccion 7.6
# El metodo connect() envia las estadisticas al conectarse:

class DashboardConsumer(AsyncWebsocketConsumer):

        async def connect(self):
        user = self.scope['user']
        if not user.is_authenticated:
                await self.close(code=4001)
                return

        self.group_name = 'dashboard'
        await self.channel_layer.group_add(self.group_name, self.channel_name)
        await self.accept()

        # Enviar estadisticas iniciales al conectarse
        stats = await self.get_stats()
        await self.send(text_data=json.dumps({
                'tipo':  'stats_iniciales',
                'stats': stats,
        }))

        async def disconnect(self, close_code):
        await self.channel_layer.group_discard(self.group_name, self.channel_name)

        async def dashboard_actualizar(self, event):
        """Recibe del channel layer y reenvia al navegador"""
        await self.send(text_data=json.dumps({
                'tipo':  'stats_actualizado',
```

```
        'stats': event['stats'],
    }))

    @database_sync_to_async
    def get_stats(self):
    from .models import Encomienda
    from django.utils import timezone
    hoy = timezone.now().date()
    return {
        'activas':   Encomienda.objects.activas().count(),
        'en_transito':   Encomienda.objects.en_transito().count(),
        'con_retraso':   Encomienda.objects.con_retraso().count(),
        'entregadas_hoy': Encomienda.objects.filter(
        estado='EN', fecha_entrega_real=hoy
        ).count(),

    }
```

Paso 4 — El modelo notifica al cambiar el estado

Ya implementado en la sección 7.8, el método `cambiar_estado()` del modelo `Encomienda` llama a `_notificar_cambio_estado()` que publica en ambos grupos: el global y el del dashboard:

Python

```
envios/models.py — _notificar_cambio_estado() (referencia)
# envios/models.py
# El metodo _notificar_cambio_estado() (ya implementado en seccion 7.8):

def _notificar_cambio_estado(self, estado_anterior, estado_nuevo, empleado):
    from django.utils import timezone
    channel_layer = get_channel_layer()

    mensaje = {
    'encomienda_id':   self.pk,
```

```
        'codigo':           self.codigo,
        'estado_anterior': estado_anterior,
        'estado_nuevo':     estado_nuevo,
        'empleado':         str(empleado),
        'timestamp':        timezone.now().isoformat(),
        }

        # Notificar al grupo global (lista de encomiendas y feed)
        async_to_sync(channel_layer.group_send)(
        'encomiendas_global',
        {'type': 'encomienda_estado_cambio', **mensaje}
        )

        # Notificar al dashboard con estadisticas actualizadas
        stats = {
        'activas':   Encomienda.objects.activas().count(),
        'en_transito':   Encomienda.objects.en_transito().count(),
        'con_retraso':   Encomienda.objects.con_retraso().count(),
        }
        async_to_sync(channel_layer.group_send)(
        'dashboard',
        {'type': 'dashboard_actualizar', 'stats': stats}

        )
```

Paso 5 — Registrar la URL del dashboard en urls.py

Python

envios/urls.py

```
# envios/urls.py
# Agregar la URL del dashboard si no existe:

from django.urls import path
from . import views
```

```
urlpatterns = [
       path('', views.dashboard, name='dashboard'),
       # ... resto de las URLs ...

]
```

Paso 6 — Verificar el ejemplo completo

```python
Verificación del ejemplo completo
# 1. Asegurarse que todos los servicios estan corriendo
docker compose ps
# web:   Up (daphne)
# db:  Up
# redis: Up

# 2. Aplicar migraciones si hay nuevas
docker compose exec web python manage.py migrate

# 3. Abrir el dashboard en el navegador
#      http://localhost:8000/
#      (redirige al dashboard si estas logueado)

# 4. Verificar en la consola del navegador (F12):
#      'WebSocket conectado al dashboard'
#      El badge debe mostrar 'EN VIVO' en verde

# 5. En otra pestana, cambiar el estado de una encomienda:
#      POST http://localhost:8000/api/v1/encomiendas/1/cambiar_estado/
#      Headers: Authorization: Bearer <tu_token>
#      Body: {"estado": "TR", "observacion": "Recogido en agencia Lima"}

# 6. En la primera pestana debes ver:
#      - El contador 'En transito' incrementa con animacion
#      - Aparece un toast: 'ENC-2026-001 Pendiente -> En transito'
```

```
#      - El evento aparece en el Feed de actividad

# 7. Tambien puedes probar desde el panel de Django Admin:
#    http://localhost:8000/admin/envios/encomienda/
#      Al guardar un cambio de estado, el dashboard se actualiza

# 8. Ver la conexion en las DevTools:
#      F12 -> Network -> WS
#      Hacer clic en la conexion ws://localhost:8000/ws/dashboard/

#      En 'Messages' se ven todos los frames en tiempo real
```

Resumen del flujo completo

Python

```
Flujo completo de notificacion WebSocket
# El flujo completo cuando un empleado cambia un estado:
#
# 1. Empleado usa la API REST o la web para cambiar el estado:
#      POST /api/v1/encomiendas/1/cambiar_estado/
#      Body: {"estado": "TR"}
#
# 2. El ViewSet llama a enc.cambiar_estado('TR', empleado, obs)
#
# 3. El modelo guarda en BD y registra en HistorialEstado
#
# 4. El modelo llama a _notificar_cambio_estado():
#    channel_layer.group_send('encomiendas_global', {...})
#      channel_layer.group_send('dashboard', {stats: {...}})
#
# 5. Django Channels distribuye a todos los consumers conectados:
#      - EncomiendaConsumer.encomienda_estado_cambio() x N empleados
#      - DashboardConsumer.dashboard_actualizar() x N empleados
#
# 6. Cada consumer envia el mensaje a su WebSocket:
```

```
#      ws.send(JSON.stringify({tipo: 'estado_cambio', ...}))
#
# 7. El navegador recibe en onmessage y actualiza la UI:
#      - Contador se anima y actualiza
#      - Toast aparece con la notificacion
#      - Feed muestra el nuevo evento
#
# Todo esto ocurre en < 100ms desde el cambio hasta la actualizacion

# en todos los navegadores conectados, sin ninguna recarga de página.
```

ARCHIVOS DEL EJEMPLO

MODIFICADOS en este ejemplo:

templates/envios/dashboard.html	<- estructura HTML + bloque extra_js con la logica JavaScript del WebSocket
envios/views.py	<- vista dashboard() con stats iniciales
envios/urls.py	<- ruta del dashboard (si no existia)

YA IMPLEMENTADOS en secciones anteriores (sin cambios):

envios/consumers.py	<- DashboardConsumer (seccion 7.6)
envios/models.py	<- _notificar_cambio_estado() (seccion 7.8)
envios/routing.py	<- ws/dashboard/ URL (seccion 7.6)
config/asgi.py	<- ProtocolTypeRouter (seccion 7.6)
config/settings.py	<- CHANNEL_LAYERS, daphne (seccion 7.6)
docker-compose.yml	<- servicio redis (seccion 7.7)

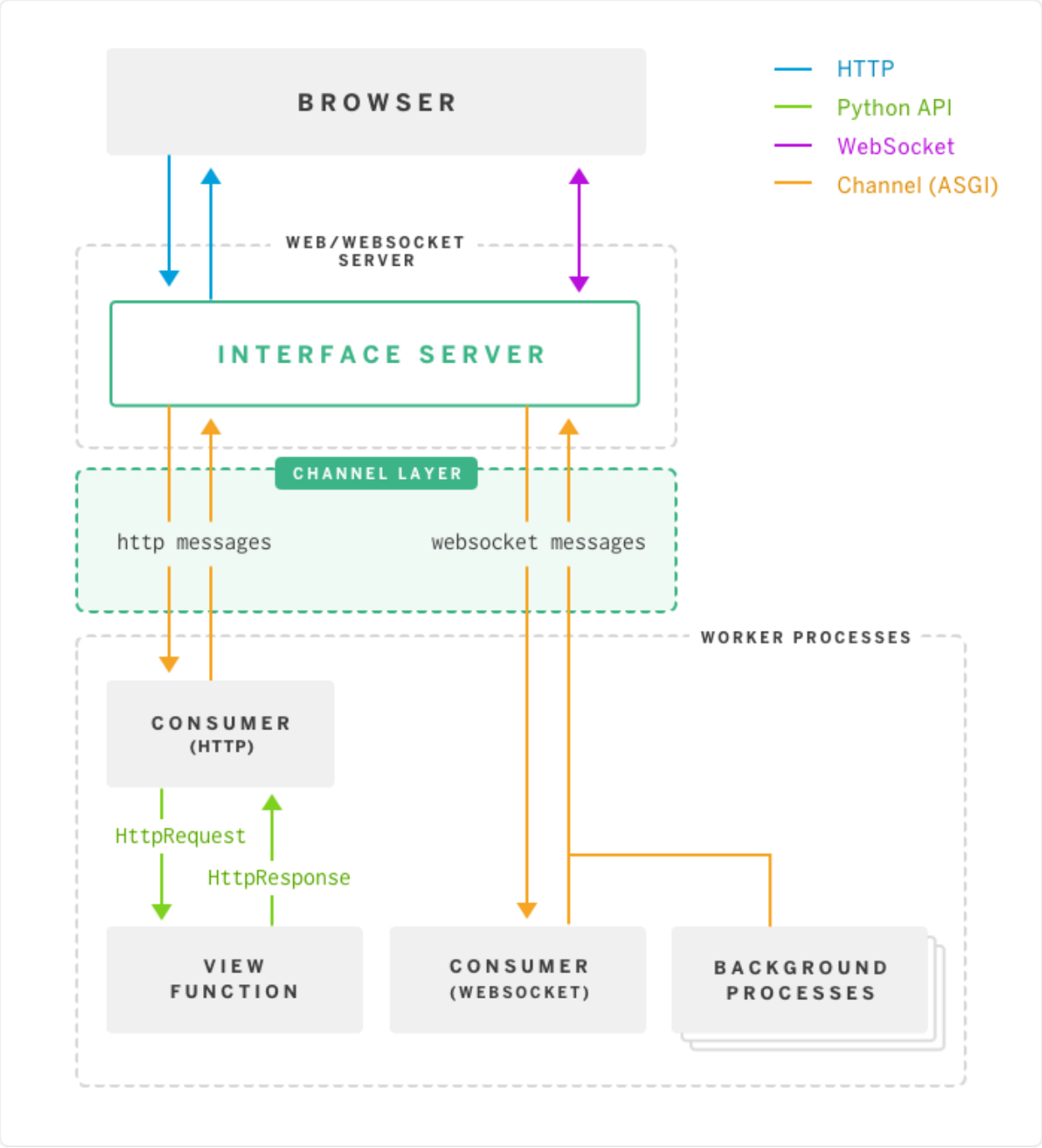
Django Channels

Django Channels es un proyecto oficial que amplía las capacidades de Django para manejar protocolos asíncronos y de larga duración, como WebSockets, MQTT y otros protocolos de mensajería, yendo más allá de las capacidades tradicionales HTTP de Django. Permite crear aplicaciones en tiempo real (chats, notificaciones, tableros en vivo) utilizando Python y la sintaxis familiar de Django

QUÉ CUBRE ESTE DOCUMENTO
1. Arquitectura de Django Channels: ASGI, channel layer, grupos, consumers
2. Tipos de consumer: AsyncWebsocketConsumer, JsonWebsocketConsumer, HttpConsumer
3. El channel layer: grupos, mensajes, broadcast y escalado con Redis
4. Autenticación y permisos en WebSockets
5. Routing: como Django Channels enruta conexiones WebSocket
6. Middleware propio para Channels
7. Testing de consumers con ChannelsLiveServerTestCase
8. Manejo de errores y reconexion automatica
9. Monitoreo y debugging en produccion

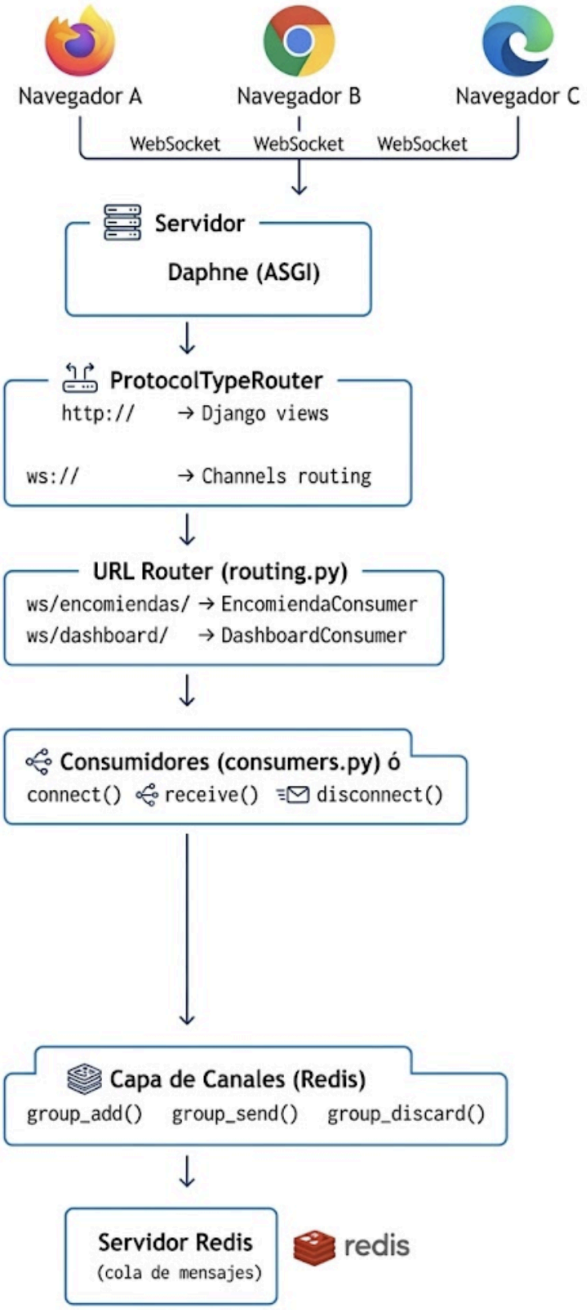
Arquitectura de Django Channels

Django Channels extiende Django añadiendo una capa asíncrona sobre ASGI. Introduce tres conceptos nuevos: **consumers** (equivalente de las vistas), el **channel layer** (bus de mensajes entre consumers), y los **grupos** (conjuntos de consumers que reciben el mismo mensaje).



1.1 Diagrama de la arquitectura completa

Arquitectura del sistema de encomiendas con Django Channels



Conceptos clave

Concepto	Equivalente en Django	Descripción en el proyecto
Consumer	Vista (View)	Clase que maneja una conexion WebSocket: connect, receive, disconnect
Channel	Hilo de ejecución	Canal único de comunicación de una conexion. Cada cliente tiene uno
Group	Sala / canal público	Conjunto de channels. 'encomiendas_global' incluye a todos los empleados
Channel Layer	Base de datos	Bus de mensajes (Redis) que conecta consumers de distintos servidores
Scope	Request object	Diccionario con información de la conexion: usuario, URL, headers
ASGI	WSGI	Protocolo asíncrono que reemplaza a WSGI para soportar WebSockets

Tipos de Consumer

En Django Channels, los consumers son la unidad básica de código y actúan como el equivalente a las vistas (views) de Django, pero para protocolos de larga duración como WebSockets.

¿Qué es un Consumer?

Un consumer es una clase en Python que "consume" eventos (mensajes, conexiones, desconexiones). A diferencia de una vista tradicional que maneja una solicitud HTTP y se cierra, un consumer permanece activo durante toda la vida de la conexión.

Funciones Principales

Los consumers gestionan el ciclo de vida de una conexión mediante métodos específicos:

- connect(): Se ejecuta cuando un cliente intenta abrir una conexión. Aquí puedes aceptar o rechazar la conexión (útil para autenticación).

- receive(): Se activa cada vez que el servidor recibe un mensaje desde el cliente.
- disconnect(): Se ejecuta cuando el cliente cierra la conexión

Tipos de Consumers

Channels ofrece clases base para facilitar el desarrollo según tus necesidades

Tipo	Descripción
`WebsocketConsumer`	Síncrono. Ideal para lógica simple que no requiere operaciones asíncronas pesadas.
`AsyncWebsocketConsumer`	Asíncrono. Recomendado para alta concurrencia y mejor rendimiento en tareas paralelas.
`JsonWebsocketConsumer`	Variante que serializa/deserializa automáticamente mensajes en formato **JSON**.

Conceptos Clave Relacionados

- Scope: Es similar al objeto request de Django. Contiene información sobre la conexión (headers, usuario, parámetros de URL).
- Channel Layer: Un sistema de mensajería (comúnmente usando Redis) que permite que diferentes instancias de consumers se comuniquen entre sí o con otras partes de Django.
- Routing: Define qué consumer debe manejar qué URL, similar a urls.py

AsyncWebsocketConsumer — el consumer principal

Es la clase base que usamos en el sistema de encomiendas. Define los tres métodos del ciclo de vida: `connect()`, `receive()` y `disconnect()`. Todos son corrutinas async.

Python

```python
envios/consumers.py — AsyncWebsocketConsumer completo
# envios/consumers.py
import json
from channels.generic.websocket import AsyncWebsocketConsumer
from channels.db import database_sync_to_async

class EncomiendaConsumer(AsyncWebsocketConsumer):
    """
    Consumer del canal global de encomiendas.
    Cada empleado conectado tiene una instancia de este consumer.
    """

    # —— connect: se ejecuta cuando el cliente abre la conexion ————
    async def connect(self):
    """
    self.scope contiene:
    - self.scope['user']       : usuario autenticado
    - self.scope['url_route']  : parametros de la URL
    - self.scope['headers']    : cabeceras HTTP del handshake
    - self.scope['path']       : ruta de la URL WebSocket
    - self.scope['query_string']: query string de la URL
    """
    user = self.scope['user']

    # 1. Verificar autenticacion
    if not user.is_authenticated:
        # close() envia un frame de cierre y rechaza la conexion
        await self.close(code=4001)
        return

    # 2. Definir a que grupo pertenece este consumer
    self.group_name = 'encomiendas_global'

    # 3. Unirse al grupo en el channel layer
    await self.channel_layer.group_add(
```

```
        self.group_name,
        self.channel_name   # <- ID unico de este consumer
)

# 4. Aceptar la conexion (enviar 101 Switching Protocols)
await self.accept()

# 5. Enviar mensaje inicial al cliente
stats = await self.get_estadisticas()
await self.send(text_data=json.dumps({
        'tipo':      'conectado',
        'usuario': user.username,
        'stats':   stats,
}))

# ── receive: se ejecuta cuando el cliente envia un mensaje ──────
async def receive(self, text_data=None, bytes_data=None):
"""
text_data:  mensaje JSON como string
bytes_data: mensaje binario (no usado en este proyecto)
"""
if not text_data:
        return

try:
        data = json.loads(text_data)
except json.JSONDecodeError:
        await self.send(text_data=json.dumps({
        'tipo': 'error', 'mensaje': 'JSON invalido'
        }))
        return

tipo = data.get('tipo')

if tipo == 'ping':
        await self.send(text_data=json.dumps({'tipo': 'pong'}))

elif tipo == 'solicitar_stats':
        stats = await self.get_estadisticas()
        await self.send(text_data=json.dumps({
        'tipo': 'stats', 'stats': stats
        }))

elif tipo == 'suscribir_encomienda':
```

```
        # Unirse al grupo especifico de una encomienda
        enc_id = data.get('encomienda_id')
        if enc_id:
        await self.channel_layer.group_add(
                f'encomienda_{enc_id}',
                self.channel_name
        )
        await self.send(text_data=json.dumps({
                'tipo': 'suscrito', 'encomienda_id': enc_id
        }))

# —— disconnect: se ejecuta cuando el cliente cierra la conexion —
async def disconnect(self, close_code):
"""
close_code: codigo de cierre WebSocket
1000 = cierre normal
1001 = el cliente navego a otra pagina
1006 = perdida de conexion de red
4001 = no autorizado (personalizado)
"""
# Salir del grupo para no recibir mas mensajes
await self.channel_layer.group_discard(
        self.group_name,
        self.channel_name
)

# —— Handler de grupo: se ejecuta cuando el channel layer envia ——
async def encomienda_estado_cambio(self, event):
"""
Se llama cuando alguien hace:
 channel_layer.group_send('encomiendas_global', {
        'type': 'encomienda_estado_cambio',  # <- nombre del metodo
        ...datos...
})
IMPORTANTE: 'type' usa puntos en lugar de underscores:
'encomienda.estado.cambio' -> encomienda_estado_cambio()
"""
await self.send(text_data=json.dumps({
        'tipo':             'estado_cambio',
        'encomienda_id':   event['encomienda_id'],
        'codigo':           event['codigo'],
        'estado_anterior': event['estado_anterior'],
        'estado_nuevo':     event['estado_nuevo'],
        'empleado':         event['empleado'],
```

```python
            'timestamp':        event['timestamp'],
        }))

        # —— Metodo auxiliar: consulta sincrona del ORM en contexto async —
        @database_sync_to_async
        def get_estadisticas(self):
        from .models import Encomienda
        return {
            'activas':   Encomienda.objects.activas().count(),
            'en_transito': Encomienda.objects.en_transito().count(),
            'con_retraso': Encomienda.objects.con_retraso().count(),

        }
```

JsonWebsocketConsumer — sin json.loads/dumps manual

Una variante de `AsyncWebsocketConsumer` que parsea y serializa JSON automáticamente. `receive_json()` recibe el dict directamente y `send_json()` serializa automáticamente.

Python

```python
AsyncJsonWebsocketConsumer
# envios/consumers.py — alternativa mas limpia con JsonWebsocketConsumer
from channels.generic.websocket import AsyncJsonWebsocketConsumer

class EncomiendaJsonConsumer(AsyncJsonWebsocketConsumer):
        """
        AsyncJsonWebsocketConsumer parsea JSON automaticamente.
        receive_json() recibe un dict en lugar de un string.
        send_json()  acepta un dict en lugar de un string.
        """

        async def connect(self):
        user = self.scope['user']
        if not user.is_authenticated:
            await self.close(code=4001)
            return
```

```
self.group_name = 'encomiendas_global'
await self.channel_layer.group_add(self.group_name, self.channel_name)
await self.accept()
stats = await self.get_estadisticas()
# send_json() serializa automaticamente el dict a JSON
await self.send_json({'tipo': 'conectado', 'stats': stats})

async def receive_json(self, content, **kwargs):
# content ya es un dict, no hay que hacer json.loads()
tipo = content.get('tipo')
if tipo == 'ping':
       await self.send_json({'tipo': 'pong'})
elif tipo == 'solicitar_stats':
       stats = await self.get_estadisticas()
       await self.send_json({'tipo': 'stats', 'stats': stats})

async def disconnect(self, close_code):
await self.channel_layer.group_discard(self.group_name, self.channel_name)

async def encomienda_estado_cambio(self, event):
# send_json() en lugar de send(text_data=json.dumps(...))
await self.send_json({
       'tipo':   'estado_cambio',
       'codigo': event['codigo'],
       'nuevo':  event['estado_nuevo'],
})

@database_sync_to_async
def get_estadisticas(self):
from .models import Encomienda

return {'activas': Encomienda.objects.activas().count()}
```

WebsocketConsumer — version sincrona (sin async)

La versión síncrona `WebsocketConsumer` se usa cuando el consumer no necesita operaciones asíncronas. Django Channels lo ejecuta en un hilo separado. En el sistema de encomiendas se prefiere la versión async.

Python

```python
WebsocketConsumer — versión síncrona (referencia)
# Solo para referencia. En el proyecto usamos la version async.
from channels.generic.websocket import WebsocketConsumer

class ConsumerSimple(WebsocketConsumer):
        """
        Version sincrona: sin async/await.
        Channels lo ejecuta en un ThreadPoolExecutor.
        Usar cuando se necesita llamar a codigo sincrono de terceros
        que no tiene soporte async.
        """
        def connect(self):
        self.accept()
         self.send(text_data='{"tipo": "conectado"}')

        def receive(self, text_data):
        data = json.loads(text_data)
         self.send(text_data=json.dumps({'tipo': 'eco', 'dato': data}))

        def disconnect(self, close_code):

        pass
```

El Channel Layer en Profundidad

El channel layer es la capa de mensajería que conecta consumers entre sí, incluso si están en distintos procesos o servidores. Redis actúa como intermediario: cuando un consumer publica en un grupo, Redis lo distribuye a todos los consumers suscritos.

Las 4 operaciones del channel layer

Python

```python
Las 4 operaciones del channel layer
# El channel layer se obtiene con get_channel_layer()
from channels.layers import get_channel_layer
channel_layer = get_channel_layer()

# 1. group_add: unir un channel a un grupo
#      Cada consumer llama esto en connect()
await channel_layer.group_add(
       'encomiendas_global',   # nombre del grupo
       self.channel_name         # ID unico del consumer
)

# 2. group_discard: quitar un channel de un grupo
#      Cada consumer llama esto en disconnect()
await channel_layer.group_discard(
       'encomiendas_global',
       self.channel_name
)

# 3. group_send: enviar un mensaje a TODOS los channels del grupo
#      Lo llama el modelo, la API REST o cualquier parte del sistema.
#      'type' indica que metodo del consumer recibe el mensaje.
await channel_layer.group_send(
       'encomiendas_global',
       {
       'type':             'encomienda_estado_cambio',  # -> handler
       'encomienda_id':   enc.pk,
       'codigo':           enc.codigo,
       'estado_anterior': anterior,
       'estado_nuevo':     nuevo,
```

```
        'empleado':         str(empleado),
        'timestamp':        timezone.now().isoformat(),
        }
)

# 4. send: enviar a UN channel especifico (no a un grupo)
#      Util para mensajes directos a un usuario especifico.
await channel_layer.send(
        'specific.channel.name',
        {'type': 'chat.message', 'message': 'Hola'}

)
```

Llamar al channel layer desde código síncrono

El modelo Django es síncrono, pero el channel layer es async. Se usa `async_to_sync()` de asgiref para llamarlo desde código síncrono como el modelo o una vista REST:

Python

```
Notificacion al channel layer desde el modelo
# envios/models.py
from asgiref.sync import async_to_sync
from channels.layers import get_channel_layer
from django.utils import timezone

class Encomienda(models.Model):

        def cambiar_estado(self, nuevo_estado, empleado, observacion=''):
        """Metodo sincrono del modelo que notifica al channel layer"""
        estado_anterior = self.estado
        self.estado = nuevo_estado
        if nuevo_estado == 'EN':
                self.fecha_entrega_real = timezone.now().date()
        self.save()

        HistorialEstado.objects.create(
```

```
        encomienda=self,
        estado_anterior=estado_anterior,
        estado_nuevo=nuevo_estado,
        empleado=empleado,
        observacion=observacion,
    )

    # async_to_sync convierte la llamada async en sincrona
    # Es seguro llamarlo desde un metodo sincrono del modelo
     self._notificar_websocket(estado_anterior, nuevo_estado, empleado)

    def _notificar_websocket(self, estado_anterior, estado_nuevo, empleado):
    channel_layer = get_channel_layer()
    if not channel_layer:
        return   # sin channel layer configurado (tests unitarios)

    mensaje = {
        'type':             'encomienda_estado_cambio',
        'encomienda_id':   self.pk,
        'codigo':           self.codigo,
        'estado_anterior': estado_anterior,
        'estado_nuevo':     estado_nuevo,
        'empleado':         str(empleado),
        'timestamp':        timezone.now().isoformat(),
    }

    # Notificar al grupo global (todos los empleados conectados)
     async_to_sync(channel_layer.group_send)('encomiendas_global', mensaje)

    # Notificar al grupo especifico de esta encomienda
     async_to_sync(channel_layer.group_send)(f'encomienda_{self.pk}', mensaje)

    # Actualizar el dashboard con las nuevas estadisticas
    stats = {
        'activas':   Encomienda.objects.activas().count(),
        'en_transito': Encomienda.objects.en_transito().count(),
        'con_retraso': Encomienda.objects.con_retraso().count(),
    }
     async_to_sync(channel_layer.group_send)(
        'dashboard',
        {'type': 'dashboard_actualizar', 'stats': stats}

    )
```

3.3 Grupos dinámicos por encomienda

Cada encomienda tiene su propio grupo dinámico. Los clientes que ven el detalle de una encomienda se suscriben a ese grupo específico y solo reciben los cambios de esa encomienda.

Python

```python
Grupos dinámicos por encomienda
# Grupos en el sistema de encomiendas:
#
# 'encomiendas_global'   <- todos los empleados conectados
#   Quien se une: EncomiendaConsumer al conectarse
#   Quien envia:  el modelo cada vez que cambia algun estado
#
# 'encomienda_42'          <- quien esta viendo el detalle de la enc. 42
#   Quien se une: EncomiendaDetalleConsumer al conectarse a ws/encomiendas/42/
#   Quien envia:  el modelo cuando cambia el estado de la enc. 42
#
# 'dashboard'              <- quien tiene el dashboard abierto
#   Quien se une: DashboardConsumer al conectarse a ws/dashboard/
#   Quien envia:  el modelo despues de cada cambio de estado

# El consumer de detalle de encomienda:
class EncomiendaDetalleConsumer(AsyncWebsocketConsumer):

       async def connect(self):
       user = self.scope['user']
       if not user.is_authenticated:
              await self.close(code=4001)
              return

       # El pk viene de la URL: ws/encomiendas/<pk>/
       self.enc_pk   = self.scope['url_route']['kwargs']['pk']
       self.group_name = f'encomienda_{self.enc_pk}'

       # Verificar que la encomienda existe antes de aceptar
       existe = await self.enc_existe(self.enc_pk)
       if not existe:
              await self.close(code=4004)   # 4004 = recurso no encontrado
              return
```

```
        await self.channel_layer.group_add(self.group_name, self.channel_name)
        await self.accept()

        # Enviar estado actual de la encomienda al conectarse
        enc_data = await self.get_encomienda(self.enc_pk)
        await self.send(text_data=json.dumps({
            'tipo':      'estado_actual',
            'encomienda': enc_data,
        }))

        async def disconnect(self, close_code):
        await self.channel_layer.group_discard(self.group_name, self.channel_name)

        async def receive(self, text_data):
        pass   # este consumer solo recibe, no procesa mensajes del cliente

        async def encomienda_estado_cambio(self, event):
        await self.send(text_data=json.dumps({
            'tipo':             'estado_cambio',
            'estado_anterior': event['estado_anterior'],
            'estado_nuevo':     event['estado_nuevo'],
            'empleado':         event['empleado'],
            'timestamp':        event['timestamp'],
        }))

        @database_sync_to_async
        def enc_existe(self, pk):
        from .models import Encomienda
        return Encomienda.objects.filter(pk=pk).exists()

        @database_sync_to_async
        def get_encomienda(self, pk):
        from .models import Encomienda
        from .serializers import EncomiendaDetailSerializer
        try:
            enc = Encomienda.objects.con_relaciones().get(pk=pk)
            return dict(EncomiendaDetailSerializer(enc).data)
        except Encomienda.DoesNotExist:

            return None
```

Routing — Cómo Channels Enruta Conexiones

El routing de Channels es el equivalente de `urls.py` de Django para WebSockets. Define qué consumer maneja cada URL WebSocket. Se usa `re_path` en lugar de `path` porque el protocolo `ws://` no soporta todos los patrones de `path()`.

envios/routing.py — completo

Python

envios/routing.py

```python
# envios/routing.py
from django.urls import re_path
from . import consumers

websocket_urlpatterns = [
        # ── Consumer general: todos los empleados conectados ─────────
        # URL: ws://localhost:8000/ws/encomiendas/
        re_path(
        r'^ws/encomiendas/$',
         consumers.EncomiendaConsumer.as_asgi(),
        name='ws-encomiendas'
        ),

        # ── Consumer de detalle: una encomienda especifica ────────────
        # URL: ws://localhost:8000/ws/encomiendas/42/
        # El pk se extrae con el grupo con nombre (?P<pk>\d+)
        re_path(
         r'^ws/encomiendas/(?P<pk>\d+)/$',
         consumers.EncomiendaDetalleConsumer.as_asgi(),
        name='ws-encomienda-detalle'
        ),

        # ── Consumer del dashboard: estadisticas en tiempo real ───────
        # URL: ws://localhost:8000/ws/dashboard/
        re_path(
        r'^ws/dashboard/$',
         consumers.DashboardConsumer.as_asgi(),
```

```
        name='ws-dashboard'
        ),
]

# Nota: re_path es necesario porque ws:// no soporta <int:pk>
# El grupo (?P<nombre>patron) captura el valor en url_route['kwargs']

# Acceso en el consumer: self.scope['url_route']['kwargs']['pk']
```

config/asgi.py — enrutador de protocolos

Python

```
config/asgi.py
# config/asgi.py
import os
import django
from django.core.asgi import get_asgi_application
from channels.routing import ProtocolTypeRouter, URLRouter
from channels.auth import AuthMiddlewareStack
from channels.security.websocket import AllowedHostsOriginValidator

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')
django.setup()

# Importar DESPUES de django.setup() para evitar errores de inicializacion
from envios.routing import websocket_urlpatterns

application = ProtocolTypeRouter({
        # Peticiones HTTP normales: Django las maneja igual que antes
        'http': get_asgi_application(),

        # Conexiones WebSocket: las maneja Channels
        'websocket': AllowedHostsOriginValidator(
        # AuthMiddlewareStack: lee la sesion/cookie de Django
        # y llena self.scope['user'] con el usuario autenticado
```

```
        AuthMiddlewareStack(
                URLRouter(websocket_urlpatterns)
        )
        ),
})

# AllowedHostsOriginValidator:
#   Rechaza conexiones WebSocket de origenes no listados en ALLOWED_HOSTS.
#   Protege contra ataques CSRF en WebSockets.

# AuthMiddlewareStack:
#   Combina SessionMiddleware + CookieMiddleware.
#   Lee la cookie de sesion de Django y popula self.scope['user'].

#   Si el usuario no esta logueado: self.scope['user'] = AnonymousUser
```

Autenticación y Permisos en WebSockets

El `AuthMiddlewareStack` del asgi.py lee la cookie de sesión de Django y llena `self.scope['user']` automáticamente. Pero cuando se usa JWT (API REST), el token no viene en una cookie sino en la cabecera `Authorization`. Se necesita un middleware personalizado.

Autenticación por sesión (para vistas web Django)

Python

```python
# Si el usuario se logueo a traves de la web de Django (no la API REST),
# AuthMiddlewareStack ya rellena self.scope['user'] automaticamente.

async def connect(self):
        user = self.scope['user']
```

```
        # user.is_authenticated es True si el usuario esta logueado
        if not user.is_authenticated:
        await self.close(code=4001)
        return

        # user es el objeto User de Django, con todos sus atributos
        print(f'Usuario conectado: {user.username}, {user.email}')

        await self.accept()
```

Autenticación por JWT — middleware personalizado

Cuando el cliente es una app móvil o un frontend React que usa JWT, el token llega como parámetro de la URL: `ws://localhost:8000/ws/encomiendas/?token=eyJhbGci...`. Se crea un middleware que lee el token y autentica al usuario:

Python

```
channels_middleware.py — autenticacion JWT
# channels_middleware.py (en la raiz del proyecto)
from channels.db import database_sync_to_async
from django.contrib.auth.models import AnonymousUser
from rest_framework_simplejwt.tokens import AccessToken
from rest_framework_simplejwt.exceptions import InvalidToken, TokenError
from django.contrib.auth import get_user_model
from urllib.parse import parse_qs

User = get_user_model()

@database_sync_to_async
def get_user_from_token(token_string):
        """
        Valida el token JWT y devuelve el usuario.
        Se ejecuta en un hilo separado (database_sync_to_async)
        porque hace consultas a la BD.
        """
```

```python
        try:
        token   = AccessToken(token_string)
        user_id = token['user_id']
        return User.objects.get(pk=user_id)
        except (InvalidToken, TokenError, User.DoesNotExist):
        return AnonymousUser()


class JWTAuthMiddleware:
        """
        Middleware de Channels que autentica al usuario via JWT.
        El token llega como parametro de la URL:
    ws://localhost:8000/ws/encomiendas/?token=eyJhbGci...
        """

        def __init__(self, inner):
        self.inner = inner

        async def __call__(self, scope, receive, send):
        # Solo procesar conexiones WebSocket
        if scope['type'] == 'websocket':
                # Extraer el token del query string de la URL
                query_string = scope.get('query_string', b'').decode('utf-8')
                params       = parse_qs(query_string)
                token_list   = params.get('token', [])

                if token_list:
                # Validar el token JWT y obtener el usuario
                scope['user'] = await get_user_from_token(token_list[0])
                else:
                scope['user'] = AnonymousUser()

        return await self.inner(scope, receive, send)


# Funcion auxiliar para usar en asgi.py
def JWTAuthMiddlewareStack(inner):

        return JWTAuthMiddleware(AuthMiddlewareStack(inner))
```

```python
Usar el middleware JWT en asgi.py
# config/asgi.py — usar el middleware JWT
from channels_middleware import JWTAuthMiddlewareStack

application = ProtocolTypeRouter({
       'http': get_asgi_application(),
       'websocket': AllowedHostsOriginValidator(
       # Reemplazar AuthMiddlewareStack por JWTAuthMiddlewareStack
       JWTAuthMiddlewareStack(
              URLRouter(websocket_urlpatterns)
       )
       ),
})

# Uso desde el cliente JavaScript (frontend React/Vue):
const token = localStorage.getItem('access_token');
const ws = new WebSocket(
  `ws://localhost:8000/ws/encomiendas/?token=${token}`

);
```

database_sync_to_async — ORM en Consumers

El ORM de Django es síncrono. Llamar a `Encomienda.objects.all()` directamente desde un consumer async bloquearía el event loop. La solución es `database_sync_to_async`, que ejecuta el código síncrono en un hilo separado sin bloquear el event loop.

6.1 Patrón correcto en consumers

Python

```python
Patrones correctos para el ORM en consumers
from channels.db import database_sync_to_async

class EncomiendaConsumer(AsyncWebsocketConsumer):

    # ── Patron 1: decorador @database_sync_to_async ──────────────
    @database_sync_to_async
    def get_encomiendas_activas(self):
    """Funcion sincrona del ORM, ejecutada en un hilo separado"""
    from .models import Encomienda
    return list(Encomienda.objects.activas().con_relaciones())

    # Uso en el consumer:
    async def receive(self, text_data):
    encs = await self.get_encomiendas_activas()
    ...


    # ── Patron 2: sync_to_async inline ──────────────────────────
    async def receive(self, text_data):
    from asgiref.sync import sync_to_async
    from .models import Encomienda

    count = await sync_to_async(
          lambda: Encomienda.objects.activas().count()
    )()


    # ── Patron 3: ORM async nativo (Django 4.1+) ────────────────
    async def receive(self, text_data):
    from .models import Encomienda

    # Equivalentes async directos:
    count = await Encomienda.objects.activas().acount()
    enc   = await Encomienda.objects.aget(pk=1)
    encs  = await Encomienda.objects.en_transito().alist()
    await enc.asave()


    # ── INCORRECTO: nunca llamar ORM sincrono directo ──────────
    async def receive_mal(self, text_data):
```

```python
from .models import Encomienda
# ESTO BLOQUEA EL EVENT LOOP:
encs = list(Encomienda.objects.all())   # SynchronousOnlyOperation

count = Encomienda.objects.count()      # SynchronousOnlyOperation
```

Testing de Consumers

Channels incluye `WebsocketCommunicator` para probar consumers sin levantar un servidor real. Es rápido y permite simular cualquier escenario de conexión, mensaje y desconexion.

Lab	**Tests de los consumers del sistema de encomiendas** Crear envios/tests/test_consumers.py

Instalacion y configuracion de tests de Channels

Python

```python
Configuración para tests
# requirements.txt
pytest-django==4.8.0
channels==4.0.0

# pytest.ini
[pytest]
DJANGO_SETTINGS_MODULE = config.settings
addopts = -v --tb=short

# config/settings.py — channel layer en memoria para tests
# (no requiere Redis corriendo durante los tests)
if 'test' in sys.argv or 'pytest' in sys.modules:
```

```
CHANNEL_LAYERS = {
'default': {
        'BACKEND': 'channels.layers.InMemoryChannelLayer',
}

}
```

Tests con WebsocketCommunicator

Python

envios/tests/test_consumers.py

```python
# envios/tests/test_consumers.py
import pytest
import json
from channels.testing import WebsocketCommunicator
from channels.layers import get_channel_layer
from config.asgi import application
from .factories import UserFactory, EncomiendaFactory, EmpleadoFactory


@pytest.mark.asyncio
@pytest.mark.django_db(transaction=True)
class TestEncomiendaConsumer:

        async def test_conexion_sin_autenticacion(self):
        """Sin autenticar: el servidor debe rechazar con codigo 4001"""
        communicator = WebsocketCommunicator(
                application,
                '/ws/encomiendas/'
        )
        connected, code = await communicator.connect()
        assert not connected
        assert code == 4001
        await communicator.disconnect()

        async def test_conexion_autenticada(self):
```

```python
    """Con usuario autenticado: el servidor acepta y envia stats"""
    from django.test import RequestFactory
    from django.contrib.auth.models import AnonymousUser

    user = await database_sync_to_async(UserFactory)()
    communicator = WebsocketCommunicator(
           application,
           '/ws/encomiendas/'
    )
    # Inyectar el usuario en el scope para simular autenticacion
    communicator.scope['user'] = user

    connected, _ = await communicator.connect()
    assert connected

    # Recibir el mensaje de bienvenida
    response = await communicator.receive_json_from(timeout=3)
    assert response['tipo'] == 'conectado'
    assert 'stats' in response
    assert 'activas' in response['stats']

    await communicator.disconnect()

    async def test_ping_pong(self):
    """El consumer responde pong al recibir ping"""
    user = await database_sync_to_async(UserFactory)()
    communicator = WebsocketCommunicator(application, '/ws/encomiendas/')
    communicator.scope['user'] = user

    await communicator.connect()
    await communicator.receive_json_from(timeout=2)  # mensaje bienvenida

    # Enviar ping
    await communicator.send_json_to({'tipo': 'ping'})

    # Recibir pong
    response = await communicator.receive_json_from(timeout=2)
    assert response['tipo'] == 'pong'

    await communicator.disconnect()

    async def test_notificacion_via_channel_layer(self):
    """El consumer recibe y reenvía mensajes del channel layer"""
    user = await database_sync_to_async(UserFactory)()
```

```
    communicator = WebsocketCommunicator(application, '/ws/encomiendas/')
    communicator.scope['user'] = user

    await communicator.connect()
    await communicator.receive_json_from(timeout=2)  # bienvenida

    # Simular que el modelo envia una notificacion al channel layer
    channel_layer = get_channel_layer()
    await channel_layer.group_send(
          'encomiendas_global',
          {
          'type':             'encomienda_estado_cambio',
          'encomienda_id':   1,
          'codigo':           'ENC-2026-001',
          'estado_anterior': 'PE',
          'estado_nuevo':     'TR',
          'empleado':         'Mendoza Cruz, Luis',
          'timestamp':        '2026-05-14T10:00:00Z',
          }
    )

    # El consumer debe recibir y reenviar al cliente
    response = await communicator.receive_json_from(timeout=3)
    assert response['tipo']          == 'estado_cambio'
    assert response['codigo']        == 'ENC-2026-001'
    assert response['estado_nuevo'] == 'TR'

    await communicator.disconnect()


# Ejecutar los tests:

# docker compose exec web pytest envios/tests/test_consumers.py -v
```

Manejo de Errores y Reconexion

Errores en el consumer

Python

```python
Manejo de errores en el consumer
# envios/consumers.py
class EncomiendaConsumer(AsyncWebsocketConsumer):

        async def receive(self, text_data):
        # Siempre envolver en try/except para evitar que la conexion
        # se cierre por un error no controlado
        try:
                data = json.loads(text_data)
                await self.procesar_mensaje(data)
        except json.JSONDecodeError:
                await self.send(text_data=json.dumps({
                'tipo':      'error',
                'codigo':  'JSON_INVALIDO',
                'mensaje': 'El mensaje no es JSON valido',
                }))
        except Exception as e:
                import logging
                logger = logging.getLogger(__name__)
                logger.error(f'Error en consumer: {e}', exc_info=True)
                await self.send(text_data=json.dumps({
                'tipo':      'error',
                'codigo':  'ERROR_INTERNO',
                'mensaje': 'Error interno del servidor',
                }))

        async def procesar_mensaje(self, data):
        tipo = data.get('tipo')
        if tipo == 'ping':
                await self.send(text_data=json.dumps({'tipo': 'pong'}))
        elif tipo == 'solicitar_stats':
                stats = await self.get_estadisticas()
```

```
            await self.send(text_data=json.dumps({'tipo': 'stats', 'stats':
stats}))
      else:
            await self.send(text_data=json.dumps({
            'tipo': 'error', 'mensaje': f'Tipo desconocido: {tipo}'

            }))
```

Reconexión automática en el cliente

```python
Reconexion con backoff exponencial (JavaScript)
// templates/base.html o templates/envios/lista.html
// Estrategia de reconexion con backoff exponencial

class EncomiendaWebSocket {
      constructor(url) {
      this.url = url;
      this.ws = null;
      this.intentos = 0;
      this.maxIntentos = 10;
      this.baseDelay = 1000;   // 1 segundo inicial
      }

      conectar() {
      if (this.ws?.readyState === WebSocket.OPEN) return;

      this.ws = new WebSocket(this.url);

      this.ws.onopen = () => {
            console.log('Conectado');
            this.intentos = 0;   // resetear contador de reintentos
           document.getElementById('ws-badge').textContent = 'EN VIVO';
           document.getElementById('ws-badge').className = 'badge bg-success';
      };
```

```
      this.ws.onmessage = (event) => {
            const data = JSON.parse(event.data);
            this.onMensaje(data);
      };

      this.ws.onclose = (event) => {
           document.getElementById('ws-badge').textContent = 'Reconectando...';
           document.getElementById('ws-badge').className = 'badge bg-warning';

            if (event.code === 4001) {
            // No autorizado: no reconectar, redirigir al login
            window.location.href = '/accounts/login/';
            return;
            }
            if (event.code === 1000) {
            // Cierre normal: no reconectar
            return;
            }
            // Backoff exponencial: 1s, 2s, 4s, 8s, ... max 30s
            const delay = Math.min(
            this.baseDelay * Math.pow(2, this.intentos),
            30000
            );
            this.intentos++;
            if (this.intentos <= this.maxIntentos) {
            console.log(`Reconectando en ${delay/1000}s (intento
${this.intentos})`);
            setTimeout(() => this.conectar(), delay);
            } else {
               document.getElementById('ws-badge').textContent = 'Desconectado';
               document.getElementById('ws-badge').className = 'badge bg-danger';
            }
      };

      this.ws.onerror = (error) => {
            console.error('WebSocket error:', error);
      };
      }

      onMensaje(data) {
      // Sobrescribir en la instancia o subclase
      console.log('Mensaje recibido:', data);
      }
```

```
        enviar(data) {
        if (this.ws?.readyState === WebSocket.OPEN) {
             this.ws.send(JSON.stringify(data));
        }
        }

        desconectar() {
        this.intentos = this.maxIntentos + 1;  // evitar reconexion
        this.ws?.close(1000, 'Desconexion manual');
        }
}

// Uso:
const wsEnc = new EncomiendaWebSocket(
        'ws://' + window.location.host + '/ws/encomiendas/'
);
wsEnc.onMensaje = (data) => {
        if (data.tipo === 'estado_cambio') mostrarToast(data);
        if (data.tipo === 'stats_actualizado') actualizarDashboard(data.stats);
};

wsEnc.conectar();
```

Estructura Final del Proyecto

```
encomiendas/
├── channels_middleware.py   <- JWTAuthMiddleware (autenticacion JWT para WS)
├── config/
│   ├── asgi.py            <- ProtocolTypeRouter, JWTAuthMiddlewareStack
│   └── settings.py        <- CHANNEL_LAYERS, InMemoryChannelLayer en tests
├── envios/
│   ├── consumers.py       <- EncomiendaConsumer, DetalleConsumer,
DashboardConsumer
│   │                          AsyncWebsocketConsumer con:
│   │                            - connect(): autenticacion + group_add + accept
│   │                            - receive(): ping, solicitar_stats, suscribir
│   │                            - disconnect(): group_discard
│   │                            - handlers de grupo: encomienda_estado_cambio
│   ├── routing.py         <- websocket_urlpatterns con 3 rutas
│   ├── models.py          <- cambiar_estado() -> notificar_websocket ()
│   │                          async_to_sync(channel_layer.group_send)
│   └── tests/
│       └── test_consumers.py  <- WebsocketCommunicator tests
├── templates/
│   └── envios/
│       ├── dashboard.html  <- WebSocket del dashboard + reconexion exponencial
│       └── lista.html      <- WebSocket de notificaciones globales
├── requirements.txt       <- channels, channels-redis, daphne
└── docker-compose.yml  <- daphne + redis
```

ENTREGABLE — DJANGO CHANNELS EN PROFUNDIDAD

Al finalizar debes poder demostrar:

Consumers

1. EncomiendaConsumer: autenticacion en connect(), 3 tipos de mensaje en receive(), group_add/discard en connect/disconnect, handler de grupo.
2. EncomiendaDetalleConsumer: grupo dinamico por pk, verificar existencia, enviar estado actual al conectarse.
3. DashboardConsumer: grupo 'dashboard', stats iniciales, handler dashboard_actualizar.
4. Todos los consumers usan @database_sync_to_async para el ORM.

Channel Layer

5. group_add/group_discard en connect/disconnect de cada consumer.
6. group_send desde el modelo con async_to_sync.
7. Los tres grupos funcionan independientemente.

Autenticacion

8. AuthMiddlewareStack llena self.scope['user'] para sesiones web.
9. JWTAuthMiddleware lee el token del query string para la API REST.
10. Conexiones sin autenticar se rechazan con codigo 4001.

Testing

11. 4 tests con WebsocketCommunicator: sin auth, conectado, ping/pong, notificacion via channel layer.
12. InMemoryChannelLayer en settings.py para tests sin Redis.

Reconexion

13. El cliente JavaScript reconecta con backoff exponencial.
14. El badge cambia a 'EN VIVO', 'Reconectando...' o 'Desconectado'.
15. La reconexion NO se ejecuta para codigos 4001 (no autorizado) ni 1000 (normal).

Redis como Channel Layer

QUÉ CUBRE ESTE DOCUMENTO
1. Qué es Redis y por qué es el channel layer recomendado para produccion
2. Cómo funciona Redis internamente como bus de mensajes (Pub/Sub)
3. Instalación y configuracion de Redis en Docker paso a paso
4. Configuracion avanzada de CHANNEL_LAYERS en settings.py
5. InMemoryChannelLayer vs RedisChannelLayer: diferencias y cuando usar cada uno
6. El problema del escalado horizontal y como Redis lo resuelve
7. Grupos en Redis: como se almacenan y expiran
8. Monitoreo de Redis: redis-cli, comandos de diagnostico
9. Persistencia: RDB, AOF y configuracion optima para Channels
10. Redis Sentinel y Redis Cluster para alta disponibilidad

Qué es Redis y por qué se usa como Channel Layer

Redis (Remote Dictionary Server) es una base de datos en memoria, de clave-valor, extremadamente rápida. Puede persistir datos en disco, soporta estructuras de datos avanzadas (listas, sets, hashes, streams) y tiene un sistema de Pub/Sub nativo que lo hace ideal como bus de mensajería.

El problema que Redis resuelve en el sistema de encomiendas

None

```
El problema del escalado horizontal
# ESCENARIO SIN REDIS: dos instancias del servidor
```

```
#
# Servidor A (proceso 1, puerto 8000)
#   - Empleado Juan conectado via WebSocket
#   - Empleado Maria conectada via WebSocket
#
# Servidor B (proceso 2, puerto 8001)
#   - Empleado Pedro conectado via WebSocket
#
# Luis cambia el estado de ENC-2026-001 (peticion llega al Servidor A)
# El modelo llama a channel_layer.group_send('encomiendas_global', ...)
#
# Sin Redis: el channel layer de A solo conoce los consumers de A
#   -> Juan recibe la notificacion  [OK]
#   -> Maria recibe la notificacion [OK]
#   -> Pedro NO recibe nada      [PROBLEMA]
#
# Con Redis: Redis es el intermediario compartido entre A y B
#   -> Juan recibe la notificacion  [OK]
#   -> Maria recibe la notificacion [OK]
#   -> Pedro recibe la notificacion [OK]
#

# Redis actua como 'pizarron comun' que todos los servidores comparten.
```

Redis vs InMemoryChannelLayer

Característica	InMemoryChannelLayer	RedisChannelLayer
Almacenamiento	RAM del proceso Python	Redis Server (proceso separado)
Escala horizontal	No: cada proceso es una isla	Sí: todos los procesos comparten Redis
Persistencia	Se pierde al reiniciar	Configurable (RDB/AOF)
Velocidad	Muy rápida (misma RAM)	Rápida (~1ms de latencia de red)
Cuándo usar	Tests y desarrollo solo	Producción y cualquier escenario real
Instalación	Incluido en channels	Requiere Redis + channels-redis
Config settings	InMemoryChannelLayer	channels_redis.core.RedisChannelLayer

Cómo Funciona Redis como Bus de Mensajes

Django Channels usa el sistema **Pub/Sub** (Publicar/Suscribir) de Redis para distribuir mensajes entre consumers. Cuando un consumer se une a un grupo, Channels crea una suscripción en Redis. Cuando el modelo llama a `group_send()`, Channels publica el mensaje en ese canal de Redis y todos los suscriptores lo reciben.

Flujo completo de un mensaje

Python

```
Flujo completo de un mensaje en el sistema de encomiendas
# Flujo cuando Luis cambia el estado de ENC-2026-001:
#
# PASO 1: Luis hace POST /api/v1/encomiendas/1/cambiar_estado/
#      {estado: 'TR', observacion: 'Recogido en agencia Lima'}
#
# PASO 2: El ViewSet llama a enc.cambiar_estado('TR', empleado)
#
# PASO 3: El modelo guarda en PostgreSQL y llama a _notificar_websocket()
#
# PASO 4: _notificar_websocket() ejecuta:
#   async_to_sync(channel_layer.group_send)(
#      'encomiendas_global',
#      {'type': 'encomienda_estado_cambio', 'codigo': 'ENC-2026-001', ...}
#   )
#
# PASO 5: channels-redis serializa el mensaje y lo publica en Redis:
#   PUBLISH asgi:group:encomiendas_global '{"codigo":"ENC-2026-001",...}'
#
# PASO 6: Redis distribuye el mensaje a todos los canales suscritos:
#   asgi:specific.channel.abc123  <- Juan (Servidor A)
#   asgi:specific.channel.def456  <- Maria (Servidor A)
#   asgi:specific.channel.ghi789  <- Pedro (Servidor B)
#
# PASO 7: Cada Daphne worker recibe el mensaje de Redis
#
# PASO 8: Channels llama al handler del consumer:
```

```
#   await consumer.encomienda_estado_cambio(event)
#
# PASO 9: El consumer envia el mensaje al WebSocket:
#   await self.send(text_data=json.dumps({...}))
#
# PASO 10: El navegador recibe el mensaje en ws.onmessage
#            y actualiza la UI con la notificacion.
```

Estructuras de datos de Redis usadas por Channels

Python

```
Estructuras de datos de Redis usadas por channels-redis
# channels-redis usa estas estructuras de Redis internamente:

# 1. KEYS (canales individuales): un canal por consumer conectado
#      Formato: asgi:specific.<channel_name>
#      Tipo: Redis List (cola FIFO)
#      TTL: expira automaticamente si no se lee en X segundos
#      Ejemplo de key real:
#    'asgi:specific.EncomiendaConsumer!6a7b8c9d...'

# 2. SETS (grupos): un set por cada grupo
#      Formato: asgi:group:<nombre_del_grupo>
#      Tipo: Redis Set
#      Contiene: todos los channel_names del grupo
#      TTL: se limpia cuando el consumer se desconecta (group_discard)
#      Ejemplo de key real:
#      'asgi:group:encomiendas_global'
#      Valor del set: {
#      'EncomiendaConsumer!abc...', <- Juan
#      'EncomiendaConsumer!def...', <- Maria
#      'EncomiendaConsumer!ghi...', <- Pedro
#      }
```

```
# Verlo en redis-cli:
docker compose exec redis redis-cli
127.0.0.1:6379> KEYS asgi:*
1) "asgi:group:encomiendas_global"
2) "asgi:group:dashboard"
3) "asgi:specific.EncomiendaConsumer!abc123"
4) "asgi:specific.DashboardConsumer!def456"

# Ver los miembros del grupo global:
127.0.0.1:6379> SMEMBERS asgi:group:encomiendas_global
1) "EncomiendaConsumer!abc123"
2) "EncomiendaConsumer!def456"
# Numero de empleados conectados = SCARD del grupo
127.0.0.1:6379> SCARD asgi:group:encomiendas_global

(integer) 2
```

Instalación y Configuración Paso a Paso

Paso 1	**Agregar Redis y channels-redis al proyecto** Modificar requirements.txt y docker-compose.yml

Abre `requirements.txt` y agrega las nuevas dependencias:

Python

```
requirements.txt
# requirements.txt
# (estas lineas ya existen)
Django==4.2
channels==4.0.0
daphne==4.0.0

# Agregar: cliente de Redis para Channels
channels-redis==4.1.0
```

```
# Agregar: cliente Python de Redis (para monitoreo directo)
redis==5.0.1
```

Paso 2	**Agregar el servicio Redis al docker-compose.yml** Redis corre como un contenedor separado

Abre `docker-compose.yml` y agrega el servicio Redis y su volumen:

Python

```
# docker-compose.yml
version: '3.9'

services:
  web:
      build: .
      command: daphne -b 0.0.0.0 -p 8000 config.asgi:application
      volumes:
      - .:/app
      ports:
      - '8000:8000'
      env_file:
      - .env
      depends_on:
      - db
      - redis      # <- el servicio web depende de Redis
      environment:
      - REDIS_URL=redis://redis:6379/1

  db:
      image: postgres:15-alpine
      volumes:
      - postgres_data:/var/lib/postgresql/data/
      environment:
      POSTGRES_DB:        ${DB_NAME}
```

```
      POSTGRES_USER:      ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}

  # ── NUEVO: servicio Redis ──────────────────────────────────────
  redis:
      image: redis:7-alpine
      ports:
      - '6379:6379' # exponer para conectar con redis-cli local
      volumes:
      - redis_data:/data
      - ./redis.conf:/usr/local/etc/redis/redis.conf  # configuracion
      command: redis-server /usr/local/etc/redis/redis.conf
      healthcheck:
      test: ['CMD', 'redis-cli', 'ping']
      interval: 10s
      timeout: 5s
      retries: 5

volumes:
  postgres_data:

  redis_data:       # <- volumen persistente para Redis
```

Paso 3	**Crear el archivo redis.conf** Configuracion de Redis optimizada para Django Channels

Crea el archivo `redis.conf` en la raíz del proyecto (junto a `docker-compose.yml`):

```python
# redis.conf
# Configuracion de Redis optimizada para Django Channels

# ── Red ──────────────────────────────────────────────────────────
bind 0.0.0.0               # escuchar en todas las interfaces
port 6379
```

```
tcp-keepalive 60          # mantener conexiones activas

# ── Memoria ──────────────────────────────────────────────
maxmemory 256mb           # limite de RAM para Redis
maxmemory-policy allkeys-lru  # cuando llega al limite: eliminar las
                          # claves menos usadas recientemente

# ── Persistencia: RDB (snapshot cada X segundos) ─────────
save 900 1                # guardar si hay 1 cambio en 900 segundos
save 300 10               # guardar si hay 10 cambios en 300 segundos
save 60 10000             # guardar si hay 10000 cambios en 60 segundos
dbfilename dump.rdb
dir /data                 # directorio del volumen Docker

# ── Logs ─────────────────────────────────────────────────
loglevel notice           # notice: mensajes importantes
logfile ''                # '' = stdout (visible en docker logs)

# ── Bases de datos ───────────────────────────────────────
databases 16              # 16 bases de datos (0-15)
                          # Channels usa la BD 1 (REDIS_URL ...redis:6379/1)

# ── Timeouts ─────────────────────────────────────────────
timeout 0                 # 0 = no cerrar conexiones inactivas

tcp-backlog 511           # cola de conexiones pendientes
```

Paso 4	**Configurar CHANNEL_LAYERS en settings.py** Conectar Django Channels con Redis

Abre `config/settings.py` y agrega la configuración del channel layer al final del archivo:

Python

```python
# config/settings.py
import sys
```

```
# —— Channel Layer con Redis (produccion y desarrollo) ——————————
# La URL se lee de la variable de entorno REDIS_URL
# En docker-compose.yml: REDIS_URL=redis://redis:6379/1
REDIS_URL = os.environ.get('REDIS_URL', 'redis://localhost:6379/1')

CHANNEL_LAYERS = {
       'default': {
       'BACKEND': 'channels_redis.core.RedisChannelLayer',
       'CONFIG': {
              # hosts: lista de URLs de Redis
              # Para un solo Redis:
              'hosts': [REDIS_URL],

              # Para Redis con password:
              # 'hosts': ['redis://:mi_password@redis:6379/1'],

              # Capacidad: max mensajes en la cola de un canal
              # Si se llena, los mensajes nuevos se descartan
              'capacity': 100,

              # Expiracion de mensajes sin leer (segundos)
              # Protege contra canales que nunca se leen
              'expiry': 60,

              # Prefijo de las claves en Redis
              # Util para compartir Redis entre proyectos
              'prefix': 'encomiendas',
       },
       },
}

# —— Channel Layer en memoria (solo para tests) ———————————————
# pytest detecta que estamos en un test y usa InMemoryChannelLayer
# para no requerir Redis corriendo durante los tests
if 'pytest' in sys.modules or 'test' in sys.argv:
       CHANNEL_LAYERS = {
       'default': {
              'BACKEND': 'channels.layers.InMemoryChannelLayer',
       }

       }
```

Paso 5	**Reconstruir y verificar la instalación** Comprobar que Redis está corriendo y conectado

Python

```python
Verificacion de la instalacion
# Reconstruir la imagen con las nuevas dependencias
docker compose down
docker compose build
docker compose up -d

# Verificar que todos los servicios estan corriendo
docker compose ps
# NAME              STATUS
# encomiendas-web   Up (healthy)
# encomiendas-db    Up (healthy)
# encomiendas-redis Up (healthy)  <- debe aparecer

# Verificar que Redis responde
docker compose exec redis redis-cli ping
# PONG

# Verificar que Django puede conectarse a Redis
docker compose exec web python manage.py shell
>>> from channels.layers import get_channel_layer
>>> from asgiref.sync import async_to_sync
>>> cl = get_channel_layer()
>>> async_to_sync(cl.group_send)(
...    'test_grupo',
...    {'type': 'test.mensaje', 'texto': 'Hola Redis!'}
... )
# Si no lanza excepcion: Redis esta correctamente conectado

# Verificar que la clave aparecio en Redis
docker compose exec redis redis-cli -n 1 KEYS 'encomiendas:*'

# 1) 'encomiendas:group:test_grupo'
```

Opciones de Configuración del Channel Layer

El bloque `CONFIG` de `CHANNEL_LAYERS` tiene varias opciones que controlan el comportamiento del channel layer. Entenderlas es importante para ajustar el sistema al volumen de mensajes del proyecto.

4.1 Tabla de opciones disponibles

Opción	Valor por defecto	Descripción
hosts	[('localhost', 6379)]	Lista de URLs o tuplas (host, puerto) de Redis
prefix	"asgi"	Prefijo para todas las claves en Redis
expiry	60	Segundos antes de que un mensaje sin leer expire
group_expiry	86400	Segundos antes de que un grupo sin actividad expire (24h)
capacity	100	Max mensajes en la cola de un canal antes de descartar
channel_capacity	{}	Capacidad por canal individual (override de capacity)
symmetric_encryption_keys	None	Claves para cifrar mensajes en Redis

Configuración completa comentada para el proyecto

Python

```python
config/settings.py — opciones completas comentadas
# config/settings.py — configuracion detallada del channel layer
CHANNEL_LAYERS = {
```

```
'default': {
'BACKEND': 'channels_redis.core.RedisChannelLayer',
'CONFIG': {
        # ── Conexion ──────────────────────────────────────────
        'hosts': [os.environ.get('REDIS_URL', 'redis://redis:6379/1')],
        # Para Redis con autenticacion:
        # 'hosts': ['redis://:password@redis:6379/1'],
        # Para Redis con SSL (produccion):
        # 'hosts': ['rediss://redis:6380/1'],

        # ── Identificacion ────────────────────────────────────
        # Prefijo para las claves en Redis.
        # Si se comparte Redis entre proyectos, usar nombre distinto.
        'prefix': 'encomiendas',
        # Las claves apareceran como: encomiendas:group:encomiendas_global

        # ── Mensajes ──────────────────────────────────────────
        # Segundos antes de que un mensaje no leido se elimine.
        # Si un consumer esta caido y no lee mensajes, estos expiran.
        'expiry': 60,

        # Max mensajes en cola de un canal.
        # Si se llena (consumer muy lento), los nuevos se descartan.
        'capacity': 100,

        # Capacidad diferente por tipo de canal:
        'channel_capacity': {
        # El dashboard puede recibir muchas actualizaciones de stats
        'ws.connect.*': 200,
        # Los canales HTTP normales con menos mensajes
        'http.request': 200,
        },

        # ── Grupos ────────────────────────────────────────────
        # Segundos antes de que un grupo inactivo se elimine.
        # 86400 = 24 horas. Los grupos se recrean al conectarse.
        'group_expiry': 86400,

        # ── Seguridad (opcional) ──────────────────────────────
        # Cifrar los mensajes en Redis.
        # Util si Redis es accesible desde fuera del servidor.
        # 'symmetric_encryption_keys': [os.environ.get('REDIS_SECRET')],
},
},
```

```
}
```

Grupos en Redis — Cómo se Almacenan

Cada grupo en Django Channels se almacena como un **Redis Set**. El set contiene los `channel_name` de todos los consumers suscritos. Al llamar a `group_send()`, channels-redis lee el set, obtiene todos los channel names, y encola el mensaje en cada uno de ellos.

- **Operaciones Redis que ejecuta Channels**

```python
Operaciones Redis de Channels
# Lo que channels-redis hace internamente al llamar a group_add():
#
# Python:  await channel_layer.group_add('encomiendas_global', channel_name)
# Redis:   SADD encomiendas:group:encomiendas_global <channel_name>
#            EXPIRE encomiendas:group:encomiendas_global 86400

# Lo que channels-redis hace internamente al llamar a group_send():
#
# Python:  await channel_layer.group_send('encomiendas_global', {...})
# Redis:
#   1. SMEMBERS encomiendas:group:encomiendas_global
#      -> {'channel_abc', 'channel_def', 'channel_ghi'}
#
#   2. Para cada channel_name del set:
#      RPUSH encomiendas:specific.channel_abc <mensaje_serializado>
#      EXPIRE encomiendas:specific.channel_abc 60
#      RPUSH encomiendas:specific.channel_def <mensaje_serializado>
#      EXPIRE encomiendas:specific.channel_def 60
#      RPUSH encomiendas:specific.channel_ghi <mensaje_serializado>
```

```
#      EXPIRE encomiendas:specific.channel_ghi 60

# Lo que channels-redis hace internamente al llamar a group_discard():
#
# Python:  await channel_layer.group_discard('encomiendas_global', channel_name)

# Redis:   SREM encomiendas:group:encomiendas_global <channel_name>
```

- **Verificar los grupos desde redis-cli**

```python
# Abrir la consola de Redis (base de datos 1 donde estan los datos de Channels)
docker compose exec redis redis-cli -n 1

# Ver todas las claves del proyecto de encomiendas
127.0.0.1:6379[1]> KEYS encomiendas:*
1) "encomiendas:group:encomiendas_global"
2) "encomiendas:group:dashboard"
3) "encomiendas:group:encomienda_42"
4) "encomiendas:specific.EncomiendaConsumer!a1b2c3"
5) "encomiendas:specific.DashboardConsumer!d4e5f6"

# Ver cuantos empleados estan conectados al canal global
127.0.0.1:6379[1]> SCARD encomiendas:group:encomiendas_global
(integer) 3   # 3 empleados conectados

# Ver los channel_names conectados al dashboard
127.0.0.1:6379[1]> SMEMBERS encomiendas:group:dashboard
1) "DashboardConsumer!d4e5f6"
2) "DashboardConsumer!g7h8i9"

# Ver el TTL de un grupo (tiempo restante antes de expirar)
127.0.0.1:6379[1]> TTL encomiendas:group:encomiendas_global
(integer) 85234   # ~23 horas restantes
```

```
# Ver el contenido de la cola de mensajes de un canal
127.0.0.1:6379[1]> LRANGE encomiendas:specific.EncomiendaConsumer!a1b2c3 0 -1
# (vacio si ya se consumieron los mensajes)

# Ver el uso de memoria de Redis
127.0.0.1:6379[1]> INFO memory
# used_memory_human: 2.50M
# maxmemory_human:   256.00M

# Salir del redis-cli

127.0.0.1:6379[1]> EXIT
```

Monitoreo de Redis en el Sistema de Encomiendas

Es importante monitorear Redis para detectar problemas: colas llenas, demasiados consumers conectados, o exceso de uso de memoria.

Comandos de diagnóstico esenciales

Python

```
Comandos de diagnóstico de Redis
# ── Informacion general de Redis ──────────────────────────────
docker compose exec redis redis-cli INFO
# Muestra: version, uptime, uso de CPU y memoria, conexiones activas,
# estadisticas de comandos, replicacion, keyspace

# ── Informacion de keyspace (estadisticas por BD) ─────────────
docker compose exec redis redis-cli INFO keyspace
# db1:keys=12,expires=10,avg_ttl=43200000
# Interprete: 12 claves en la BD 1, 10 con TTL activo

# ── Clientes conectados ───────────────────────────────────────
```

```
docker compose exec redis redis-cli INFO clients
# connected_clients:5
# Cuantas conexiones tiene Redis abiertas (Daphne workers + cli)

# ── Uso de memoria ──────────────────────────────────────────
docker compose exec redis redis-cli INFO memory
# used_memory_human:2.50M
# maxmemory_human:256.00M
# Si used_memory se acerca a maxmemory, redis elimina claves (LRU)

# ── Monitor en tiempo real: ver todos los comandos ──────────────
docker compose exec redis redis-cli MONITOR
# OK
# 1715684400.123456 [1 127.0.0.1:56789] "SADD"
"encomiendas:group:encomiendas_global" "abc"
# 1715684401.456789 [1 127.0.0.1:56789] "RPUSH" "encomiendas:specific.abc" "..."
# Ctrl+C para detener. ADVERTENCIA: en produccion puede generar mucho output

# ── Latencia de comandos ────────────────────────────────────
docker compose exec redis redis-cli --latency
# min: 0, max: 1, avg: 0.09 (samples: 2500)

# La latencia ideal es < 1ms en un entorno local
```

Monitoreo desde Python — vista de salud

Agrega un endpoint de salud del sistema al proyecto para verificar Redis desde la API:

Python

```python
envios/views.py — endpoint de salud del sistema
# envios/views.py — endpoint de salud del sistema
import redis
from django.http import JsonResponse
from django.conf import settings

def health_check(request):
    """
    GET /health/
```

```
Verifica que todos los servicios del sistema esten funcionando.
Incluye el estado de Redis y del channel layer.
"""
estado = {
'postgres': False,
'redis':     False,
'channels': False,
}

# Verificar PostgreSQL
try:
from django.db import connection
connection.ensure_connection()
estado['postgres'] = True
except Exception as e:
estado['postgres_error'] = str(e)

# Verificar Redis directamente
try:
r = redis.from_url(
       settings.REDIS_URL,
       socket_connect_timeout=2,
       socket_timeout=2,
)
r.ping()
info = r.info()
estado['redis']            = True
estado['redis_memoria']   = info.get('used_memory_human')
estado['redis_clientes']  = info.get('connected_clients')
estado['redis_version']   = info.get('redis_version')
except Exception as e:
estado['redis_error'] = str(e)

# Verificar Channel Layer (publicar y recibir un mensaje de prueba)
try:
from channels.layers import get_channel_layer
from asgiref.sync import async_to_sync
cl = get_channel_layer()
async_to_sync(cl.group_send)(
       'health_check',
       {'type': 'health.ping'}
)
estado['channels'] = True
except Exception as e:
```

```
        estado['channels_error'] = str(e)

        # Contar empleados conectados
        try:
        r = redis.from_url(settings.REDIS_URL)
        estado['empleados_conectados'] = r.scard(
              'encomiendas:group:encomiendas_global'
        )
        except Exception:
        estado['empleados_conectados'] = None

        todo_ok = all([estado['postgres'], estado['redis'], estado['channels']])
        http_status = 200 if todo_ok else 503
        return JsonResponse(estado, status=http_status)

# envios/urls.py — registrar el endpoint
urlpatterns += [
        path('health/', views.health_check, name='health'),

]
```

Persistencia en Redis

Redis es una base de datos en memoria, pero puede persistir datos en disco para recuperarse ante reinicios. Django Channels usa Redis para colas de mensajes, no para datos permanentes, por lo que la persistencia es opcional pero recomendada para no perder mensajes pendientes.

RDB vs AOF

Mecanismo	Cómo funciona	Ventaja	Desventaja
RDB (snapshot)	Guarda una foto de toda la BD en disco cada X segundos	Archivo compacto, recuperacion rapida	Puede perder hasta X segundos de datos
AOF (append-only file)	Guarda cada comando de escritura en un log	Perdida de datos minima	Archivo mas grande,

			recuperacion mas lenta
Ninguno	Solo en memoria, sin persistencia	Maxima velocidad	Pierde todo al reiniciar

7.2 Configuración recomendada para el proyecto

```python
redis.conf — persistencia optimizada
# redis.conf — persistencia recomendada para Django Channels

# ── RDB: snapshot en disco ──────────────────────────────────────
# Guardar si hay al menos 1 cambio en los ultimos 15 minutos
save 900 1
# Guardar si hay al menos 10 cambios en los ultimos 5 minutos
save 300 10
# Guardar si hay al menos 10000 cambios en el ultimo minuto
save 60 10000

# Nombre del archivo RDB
dbfilename encomiendas-dump.rdb
dir /data

# Si el guardado en disco falla, rechazar escrituras (seguridad)
stop-writes-on-bgsave-error yes

# Comprimir el archivo RDB
rdbcompression yes

# ── Comentario sobre AOF para Channels ──────────────────────────
# Para Django Channels NO se recomienda AOF:
# Los mensajes del channel layer son efimeros (expiran en 60s).
# Un AOF de mensajes de WebSocket crece rapidamente y no aporta valor.
# El RDB es suficiente: si Redis se reinicia, los consumers
# se reconectan y generan nuevos grupos automaticamente.

appendonly no
```

Alta Disponibilidad con Redis Sentinel

Para producción con alta disponibilidad, Redis Sentinel monitorea el servidor Redis principal y promueve automáticamente una réplica sí el principal falla. `channels-redis` soporta Sentinel de forma nativa.

Configurar Channels con Redis Sentinel

Python

```
Redis Sentinel para alta disponibilidad
# docker-compose.yml — con Redis Sentinel (produccion)
services:
  redis-master:
      image: redis:7-alpine
      command: redis-server --port 6379
      volumes:
      - redis_master_data:/data

  redis-replica:
      image: redis:7-alpine
      command: redis-server --port 6379 --replicaof redis-master 6379
      depends_on: [redis-master]

  redis-sentinel:
      image: redis:7-alpine
      command: >
      redis-sentinel /etc/redis/sentinel.conf
      --sentinel monitor mymaster redis-master 6379 1
      --sentinel down-after-milliseconds mymaster 5000
      --sentinel failover-timeout mymaster 10000
      depends_on: [redis-master, redis-replica]
```

```
# config/settings.py - usar Sentinel
CHANNEL_LAYERS = {
       'default': {
       'BACKEND': 'channels_redis.core.RedisChannelLayer',
       'CONFIG': {
              'hosts': [
              {
                     'sentinels': [('redis-sentinel', 26379)],
                     'master_name': 'mymaster',
                     'sentinel_kwargs': {},
                     'db': 1,
              }
              ],
              'prefix': 'encomiendas',
       },
       }

}
```

Problemas Comunes y Soluciones

Problema	Causa probable	Solución
ConnectionRefusedError al iniciar	Redis no esta corriendo	docker compose up -d redis && verificar con redis-cli ping
Los consumers no reciben mensajes	Prefijo incorrecto en CHANNEL_LAYERS	Verificar que 'prefix' coincide en settings.py y redis.conf
Mensajes se descartan (capacity exceeded)	Consumer muy lento o 'capacity' muy bajo	Aumentar 'capacity', revisar el handler del consumer
Grupos no se limpian (memory leak)	group_discard() no se llama en disconnect	Asegurarse de llamar group_discard en disconnect()

Redis usa demasiada memoria	Demasiados canales o mensajes pendientes	Revisar 'maxmemory' y 'maxmemory-policy' en redis.conf
Latencia alta en notificaciones	Redis en servidor distinto o sobrecargado	Colocar Redis cerca del servidor web, revisar INFO latency
django.core.exceptions.ImproperlyConfigured	CHANNEL_LAYERS no configurado en settings	Agregar el bloque CHANNEL_LAYERS completo a settings.py

Diagnosticar un problema de mensajes perdidos

Python

```python
Diagnostico de mensajes perdidos
# Si los WebSockets no reciben mensajes, seguir estos pasos:

# 1. Verificar que Redis esta corriendo
docker compose exec redis redis-cli ping
# Si no responde: docker compose start redis

# 2. Verificar que el channel layer funciona desde Django
docker compose exec web python manage.py shell
>>> from channels.layers import get_channel_layer
>>> from asgiref.sync import async_to_sync
>>> cl = get_channel_layer()
>>> print(cl)   # debe mostrar RedisChannelLayer, no InMemoryChannelLayer
>>> async_to_sync(cl.group_send)('encomiendas_global', {'type': 'test'})
# Si lanza excepcion: problema de conexion a Redis

# 3. Verificar que los grupos existen en Redis
docker compose exec redis redis-cli -n 1
KEYS encomiendas:group:*
# Si no aparece 'encomiendas:group:encomiendas_global':
# ningun consumer esta conectado, o el prefijo es incorrecto

# 4. Verificar el prefijo
```

```
docker compose exec redis redis-cli -n 1 KEYS '*:group:*'
# Si aparece 'asgi:group:...' en lugar de 'encomiendas:group:...',
# el prefix en settings.py no esta siendo aplicado correctamente

# 5. Ver los logs de Daphne en tiempo real
docker compose logs -f web

# Buscar errores de Redis o de Channels
```